

# OmniLitter

Sistema multiplataforma para la gestión y análisis de datos de  
contaminación por residuos



Escuela Técnica Superior de  
**Ingeniería Informática**

**Trabajo Fin de Grado**

**Grado:** Ingeniería Informática - Ingeniería del Software

**Centro:** ETSII — Universidad de Sevilla

**Curso académico:** 2025/26

**Autores:**

- Alejandro Castilla — alecasbar@alum.us.es
- Ignacio Martínez Díaz — ignmardial@alum.us.es

**Tutor académico:** Cruz Mata, Fermín

**Cotutor:** Ortega Rodríguez, Francisco Javier

**Cliente / entidad colaboradora:** Daniel González Fernández /  
Universidad de Cádiz

# Resumen

# Abstract

# Índice general

|                                                          |          |
|----------------------------------------------------------|----------|
| <b>1. Introducción</b>                                   | <b>1</b> |
| 1.1. Contexto y motivación . . . . .                     | 1        |
| 1.2. Problema abordado . . . . .                         | 2        |
| 1.3. Objetivos . . . . .                                 | 2        |
| 1.4. Alcance del proyecto . . . . .                      | 3        |
| <b>2. Contexto y antecedentes</b>                        | <b>5</b> |
| 2.1. Monitorización de residuos . . . . .                | 5        |
| 2.2. Aplicaciones de campo y ciencia ciudadana . . . . . | 5        |
| 2.3. Herramientas existentes . . . . .                   | 6        |
| 2.4. Taxonomías y datos científicos . . . . .            | 6        |
| 2.5. Conclusiones del análisis . . . . .                 | 7        |
| <b>3. Descripción del proyecto</b>                       | <b>8</b> |
| 3.1. Visión general de OmniLitter . . . . .              | 8        |
| 3.2. Actores y perfiles de usuario . . . . .             | 8        |
| 3.2.1. Visitante . . . . .                               | 9        |
| 3.2.2. Usuario autenticado . . . . .                     | 9        |
| 3.2.3. Investigador de campo . . . . .                   | 9        |
| 3.2.4. Administrador de grupo . . . . .                  | 10       |
| 3.2.5. Superadministrador . . . . .                      | 10       |
| 3.2.6. Usuario sin grupo activo . . . . .                | 10       |
| 3.2.7. Actores de apoyo . . . . .                        | 11       |
| 3.3. Plataformas del sistema . . . . .                   | 11       |
| 3.3.1. Aplicación móvil . . . . .                        | 11       |
| 3.3.2. Portal web . . . . .                              | 12       |
| 3.3.3. Backend y almacenamiento centralizado . . . . .   | 12       |
| 3.3.4. Integración de las plataformas . . . . .          | 12       |
| 3.4. Funcionalidades principales . . . . .               | 13       |
| 3.4.1. Gestión de usuarios y grupos . . . . .            | 13       |
| 3.4.2. Gestión de campañas . . . . .                     | 13       |
| 3.4.3. Sesiones de muestreo . . . . .                    | 13       |

|           |                                                                       |           |
|-----------|-----------------------------------------------------------------------|-----------|
| 3.4.4.    | Registro de observaciones . . . . .                                   | 13        |
| 3.4.5.    | Clasificación mediante taxonomías . . . . .                           | 14        |
| 3.4.6.    | Registro fotográfico y geolocalización . . . . .                      | 14        |
| 3.4.7.    | Trabajo offline y sincronización . . . . .                            | 14        |
| 3.4.8.    | Consulta y visualización de información . . . . .                     | 14        |
| 3.4.9.    | Exportación de datos . . . . .                                        | 14        |
| 3.4.10.   | Resumen funcional . . . . .                                           | 15        |
| 3.5.      | Restricciones y alcance excluido . . . . .                            | 15        |
| <b>4.</b> | <b>Requisitos y análisis</b>                                          | <b>17</b> |
| 4.1.      | Evolución del registro de requisitos . . . . .                        | 17        |
| 4.1.1.    | Versión 1: toma inicial de requisitos . . . . .                       | 17        |
| 4.1.2.    | Versión 2: validación mediante prototipos . . . . .                   | 19        |
| 4.1.3.    | Versión 3: consolidación tras la última reunión y revisión de dominio | 20        |
| 4.2.      | Casos de uso de referencia . . . . .                                  | 21        |
| 4.3.      | Requisitos finales consolidados . . . . .                             | 22        |
| 4.3.1.    | CU-01 – Autenticarse en el sistema . . . . .                          | 22        |
| 4.3.2.    | CU-02 – Seleccionar grupo activo . . . . .                            | 23        |
| 4.3.3.    | CU-03 – Crear sesión de muestreo . . . . .                            | 24        |
| 4.3.4.    | CU-04 – Registrar observación . . . . .                               | 25        |
| 4.3.5.    | CU-05 – Revisar y finalizar sesión . . . . .                          | 26        |
| 4.3.6.    | CU-06 – Sincronizar datos pendientes . . . . .                        | 26        |
| 4.3.7.    | CU-07 – Consultar sesiones y observaciones . . . . .                  | 27        |
| 4.3.8.    | CU-08 – Administrar usuarios y grupos . . . . .                       | 28        |
| 4.3.9.    | CU-09 – Exportar datos . . . . .                                      | 29        |
| 4.3.10.   | CU-10 – Desplegar y operar el sistema . . . . .                       | 29        |
| 4.4.      | Requisitos no funcionales consolidados . . . . .                      | 30        |
| 4.5.      | Requisitos de datos científicos consolidados . . . . .                | 31        |
| 4.6.      | Trazabilidad final . . . . .                                          | 32        |
| <b>5.</b> | <b>Planificación y gestión</b>                                        | <b>35</b> |
| 5.1.      | Metodología de trabajo . . . . .                                      | 35        |
| 5.2.      | Herramientas de gestión . . . . .                                     | 35        |
| 5.3.      | Backlog y organización por sprints . . . . .                          | 36        |
| 5.4.      | Gestión de reuniones . . . . .                                        | 45        |
| 5.5.      | Gestión de riesgos . . . . .                                          | 45        |
| 5.6.      | Gestión del alcance . . . . .                                         | 45        |
| <b>6.</b> | <b>Tecnologías utilizadas</b>                                         | <b>46</b> |
| 6.1.      | Criterios de selección . . . . .                                      | 46        |
| 6.2.      | Portal web . . . . .                                                  | 47        |
| 6.3.      | Aplicación móvil . . . . .                                            | 48        |
| 6.4.      | Backend y persistencia . . . . .                                      | 48        |

|           |                                                                       |           |
|-----------|-----------------------------------------------------------------------|-----------|
| 6.5.      | Despliegue y herramientas auxiliares . . . . .                        | 49        |
| 6.6.      | Alternativas consideradas . . . . .                                   | 49        |
| <b>7.</b> | <b>Diseño de la solución</b>                                          | <b>50</b> |
| 7.1.      | Arquitectura general . . . . .                                        | 50        |
| 7.1.1.    | Justificación de las decisiones arquitectónicas principales . . . . . | 51        |
| 7.2.      | Diseño de la base de datos . . . . .                                  | 51        |
| 7.2.1.    | Modelo relacional . . . . .                                           | 51        |
| 7.2.2.    | Descripción de entidades . . . . .                                    | 55        |
| 7.2.3.    | Decisiones de diseño de la base de datos . . . . .                    | 57        |
| 7.3.      | Diseño de la API REST . . . . .                                       | 59        |
| 7.4.      | Seguridad y control de acceso . . . . .                               | 62        |
| 7.5.      | Diseño del portal web . . . . .                                       | 62        |
| 7.5.1.    | Arquitectura de componentes . . . . .                                 | 62        |
| 7.5.2.    | Gestión de estado . . . . .                                           | 62        |
| 7.5.3.    | Flujo de navegación y control de acceso por rol . . . . .             | 63        |
| 7.6.      | Diseño de la aplicación móvil . . . . .                               | 63        |
| 7.6.1.    | Stack tecnológico y estructura de pantallas . . . . .                 | 63        |
| 7.6.2.    | Gestión de estado global . . . . .                                    | 63        |
| 7.6.3.    | Base de datos local con SQLite . . . . .                              | 64        |
| 7.6.4.    | Flujo de captura de una observación . . . . .                         | 64        |
| 7.7.      | Sincronización offline . . . . .                                      | 64        |
| 7.7.1.    | Patrón Outbox Queue . . . . .                                         | 64        |
| 7.7.2.    | Estados de sincronización . . . . .                                   | 65        |
| 7.7.3.    | Resolución de conflictos . . . . .                                    | 65        |
| 7.7.4.    | Garantía de identificadores únicos en entorno offline . . . . .       | 66        |
| <b>8.</b> | <b>Desarrollo, implantación y validación</b>                          | <b>67</b> |
| 8.1.      | Organización del desarrollo . . . . .                                 | 67        |
| 8.1.1.    | Metodología de trabajo . . . . .                                      | 67        |
| 8.1.2.    | Gestión del proyecto en GitHub Projects . . . . .                     | 67        |
| 8.1.3.    | Plan de pruebas . . . . .                                             | 67        |
| 8.1.4.    | Criterios de finalización de tareas . . . . .                         | 69        |
| 8.1.5.    | Estructura de releases . . . . .                                      | 69        |
| 8.2.      | Sprint 1: Base funcional del sistema . . . . .                        | 70        |
| 8.2.1.    | Objetivo del sprint . . . . .                                         | 70        |
| 8.2.2.    | Parte del proyecto abordada . . . . .                                 | 70        |
| 8.2.3.    | Tiempo invertido y reparto del trabajo . . . . .                      | 70        |
| 8.2.4.    | Implementación realizada . . . . .                                    | 70        |
| 8.2.5.    | Pruebas realizadas . . . . .                                          | 70        |
| 8.2.6.    | Desviaciones e incidencias . . . . .                                  | 71        |
| 8.2.7.    | Resultado del sprint . . . . .                                        | 71        |
| 8.3.      | Sprint 2: Captura de campo y análisis inicial . . . . .               | 72        |

|            |                                                                    |           |
|------------|--------------------------------------------------------------------|-----------|
| 8.3.1.     | Objetivo del sprint . . . . .                                      | 72        |
| 8.3.2.     | Parte del proyecto abordada . . . . .                              | 72        |
| 8.3.3.     | Tiempo invertido y reparto del trabajo . . . . .                   | 72        |
| 8.3.4.     | Implementación realizada . . . . .                                 | 72        |
| 8.3.5.     | Pruebas realizadas . . . . .                                       | 72        |
| 8.3.6.     | Desviaciones e incidencias . . . . .                               | 73        |
| 8.3.7.     | Resultado del sprint . . . . .                                     | 73        |
| 8.4.       | Sprint 3: Sincronización, mapas y exportación científica . . . . . | 74        |
| 8.4.1.     | Objetivo del sprint . . . . .                                      | 74        |
| 8.4.2.     | Parte del proyecto abordada . . . . .                              | 74        |
| 8.4.3.     | Tiempo invertido y reparto del trabajo . . . . .                   | 74        |
| 8.4.4.     | Implementación realizada . . . . .                                 | 74        |
| 8.4.5.     | Pruebas realizadas . . . . .                                       | 74        |
| 8.4.6.     | Desviaciones e incidencias . . . . .                               | 75        |
| 8.4.7.     | Resultado del sprint . . . . .                                     | 75        |
| 8.5.       | Sprint 4: Cierre funcional y refinamiento . . . . .                | 76        |
| 8.5.1.     | Objetivo del sprint . . . . .                                      | 76        |
| 8.5.2.     | Parte del proyecto abordada . . . . .                              | 76        |
| 8.5.3.     | Tiempo invertido y reparto del trabajo . . . . .                   | 76        |
| 8.5.4.     | Implementación realizada . . . . .                                 | 76        |
| 8.5.5.     | Pruebas realizadas . . . . .                                       | 76        |
| 8.5.6.     | Desviaciones e incidencias . . . . .                               | 77        |
| 8.5.7.     | Resultado del sprint . . . . .                                     | 77        |
| 8.6.       | Resumen global del desarrollo . . . . .                            | 78        |
| 8.6.1.     | Funcionalidades implementadas . . . . .                            | 78        |
| 8.6.2.     | Evolución del sistema . . . . .                                    | 78        |
| 8.6.3.     | Cobertura de requisitos . . . . .                                  | 78        |
| 8.6.4.     | Matriz de trazabilidad prueba/requisito . . . . .                  | 78        |
| 8.6.5.     | Limitaciones finales . . . . .                                     | 78        |
| <b>9.</b>  | <b>Despliegue y operación</b>                                      | <b>79</b> |
| 9.1.       | Arquitectura de despliegue . . . . .                               | 79        |
| 9.2.       | Configuración de producción . . . . .                              | 79        |
| 9.3.       | Despliegue de API y base de datos . . . . .                        | 79        |
| 9.4.       | Despliegue del portal web . . . . .                                | 79        |
| 9.5.       | Build y distribución de la aplicación móvil . . . . .              | 79        |
| 9.6.       | Limitaciones operativas . . . . .                                  | 79        |
| <b>10.</b> | <b>Conclusiones y trabajo futuro</b>                               | <b>80</b> |
| 10.1.      | Cumplimiento de objetivos . . . . .                                | 80        |
| 10.2.      | Limitaciones . . . . .                                             | 80        |
| 10.3.      | Lecciones aprendidas . . . . .                                     | 80        |
| 10.4.      | Líneas de trabajo futuro . . . . .                                 | 80        |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>A. Declaración del uso de IA</b>                       | <b>83</b> |
| A.1. Usos realizados . . . . .                            | 83        |
| A.2. Ejemplos de prompts utilizados . . . . .             | 84        |
| A.3. Autoría y revisión crítica . . . . .                 | 85        |
| <b>B. Requisitos detallados y validación con cliente</b>  | <b>86</b> |
| B.1. Resumen de reuniones con el cliente . . . . .        | 86        |
| B.2. Requisitos obtenidos en la reunión inicial . . . . . | 86        |
| B.3. Validación de requisitos mediante mockups . . . . .  | 87        |
| B.4. Requisitos funcionales detallados . . . . .          | 88        |
| B.5. Requisitos no funcionales detallados . . . . .       | 88        |
| B.6. Requisitos de datos científicos . . . . .            | 88        |
| <b>C. Manual de instalación y despliegue</b>              | <b>90</b> |
| <b>D. Manual de usuario</b>                               | <b>91</b> |
| <b>E. Pruebas completas</b>                               | <b>92</b> |
| <b>F. API y diccionario de datos</b>                      | <b>93</b> |
| <b>G. Backlog completo</b>                                | <b>94</b> |

# Índice de figuras

|                                                                                                                         |    |
|-------------------------------------------------------------------------------------------------------------------------|----|
| 7.1. Arquitectura general propuesta para OmniLitter. . . . .                                                            | 51 |
| 7.2. Modelo entidad-relación. Bloque de usuarios, grupos, campañas y catálogo de entornos. . . . .                      | 53 |
| 7.3. Modelo entidad-relación. Bloque de sesiones de muestreo, observaciones, residuos, fotografías y taxonomía. . . . . | 54 |
| 7.4. Modelo entidad-relación. Bloque de autenticación y seguridad. . . . .                                              | 54 |

# Índice de cuadros

|      |                                                                                 |    |
|------|---------------------------------------------------------------------------------|----|
| 4.1. | Requisitos identificados en la versión 1. . . . .                               | 18 |
| 4.2. | Requisitos definidos en la versión 2 tras la validación con prototipos. . . . . | 19 |
| 4.3. | Requisitos consolidados o refinados en la versión 3. . . . .                    | 21 |
| 5.1. | Resumen del backlog inicial integrado en el Sprint 1. . . . .                   | 36 |
| 5.2. | Resumen del backlog del Sprint 2: captura avanzada y flujo offline. . . . .     | 42 |
| 6.1. | Tecnologías empleadas en los prototipos funcionales. . . . .                    | 47 |
| 6.2. | Stack seleccionado para el portal web. . . . .                                  | 47 |
| 6.3. | Stack seleccionado para la aplicación móvil. . . . .                            | 48 |
| 6.4. | Stack seleccionado para backend y persistencia. . . . .                         | 49 |
| 7.1. | Descripción de entidades del modelo de datos. . . . .                           | 55 |
| 7.2. | Principales <i>endpoints</i> de la API REST de OmniLitter. . . . .              | 59 |
| 7.3. | Estados de sincronización y su significado. . . . .                             | 65 |
| 8.1. | Estrategia general de pruebas del proyecto . . . . .                            | 68 |
| 8.2. | Alcance real de las pruebas automatizadas . . . . .                             | 68 |
| 8.3. | Leyenda de códigos usados en las pruebas . . . . .                              | 69 |
| 8.4. | Resultado global de la ejecución de pruebas automatizadas . . . . .             | 69 |
| 8.5. | Pruebas automatizadas vinculadas al Sprint 1 . . . . .                          | 70 |
| 8.6. | Pruebas automatizadas vinculadas al Sprint 2 . . . . .                          | 72 |
| 8.7. | Pruebas automatizadas vinculadas al Sprint 3 . . . . .                          | 74 |
| 8.8. | Pruebas automatizadas vinculadas al Sprint 4 . . . . .                          | 76 |
| A.1. | Detalle del uso de IA en el trabajo. . . . .                                    | 84 |
| B.1. | Resumen de reuniones mantenidas con el cliente. . . . .                         | 86 |

# 1. Introducción

## 1.1. Contexto y motivación

---

La contaminación por residuos en entornos terrestres y acuáticos constituye uno de los principales problemas del medio ambiente a nivel mundial. La acumulación de residuos en los distintos tipos de entornos genera un impacto negativo sobre los ecosistemas, la biodiversidad y la calidad de vida de las personas. Con el objetivo de comprender la magnitud del problema y desarrollar estrategias eficaces de prevención y mitigación, organizaciones, administraciones públicas e instituciones científicas impulsan programas de monitorización y análisis de residuos en diferentes compartimentos ambientales.

La utilidad de estos programas depende de que los datos recogidos sean comparables, estén correctamente georreferenciados y conserven información suficiente sobre cómo, cuándo y dónde se han obtenido. En el caso de las basuras fluviales, el Joint Research Centre de la Comisión Europea señala la falta de información sistemática sobre la cantidad de residuos transportados por los ríos y la ausencia de metodologías armonizadas capaces de generar datos cuantitativos comparables [3]. De forma similar, trabajos recientes subrayan la necesidad de armonizar las técnicas de monitorización de macroplásticos en ecosistemas acuáticos y terrestres [2].

La obtención de datos reales y útiles requiere la participación de diferentes personas y grupos dispuestos a realizar campañas de monitorización en los entornos afectados. No obstante, los registros manuales, las hojas de cálculo aisladas o las campañas de limpieza sin un protocolo común suelen producir información dispersa, difícil de comparar y con metadatos insuficientes para un análisis científico posterior.

Teniendo en cuenta lo anterior, el cliente ha planteado la necesidad de desarrollar una herramienta que solucione el problema.

En respuesta a esta necesidad, se propone el desarrollo de OmniLitter. Esta plataforma incluirá tanto un portal web como una aplicación móvil, para la monitorización y registro de residuos para un posterior análisis.

## 1.2. Problema abordado

---

Para hacer un seguimiento de los residuos se necesita que diferentes personas colaboren, que se realicen campañas en lugares diversos y que se recoja información suficiente para analizar patrones espaciales y temporales. Si no se dispone de herramientas especializadas, aparecen problemas de información dispersa, falta de orden en los registros y dificultad para integrar los datos obtenidos por distintos participantes.

Generalmente, se recoge la información en la naturaleza, donde no siempre hay conexión a Internet. Esto limita el uso de aplicaciones que necesitan estar siempre conectadas a servidores remotos. Por lo tanto, es necesario tener mecanismos que permitan guardar la información en el dispositivo y sincronizarla después cuando haya conexión a la red.

El proyecto también requiere el uso de clasificaciones de residuos estandarizadas, la asociación de fotos a cada observación, el registro preciso de la ubicación geográfica y la captura de metadatos de muestreo, como protocolo, entorno, duración, transecto o área observada. Hacer todo esto a mano aumenta el riesgo de errores y dificulta el uso posterior de los datos.

El cliente necesita una solución de software que permita gestionar campañas de seguimiento, registrar observaciones geolocalizadas desde dispositivos móviles, operar sin conexión a Internet y centralizar toda la información recopilada en una infraestructura común accesible desde un portal web.

## 1.3. Objetivos

---

El objetivo principal de este Trabajo Fin de Grado es diseñar, desarrollar e implementar una plataforma software denominada OmniLitter que facilite la recogida y el posterior análisis de datos relacionados con la monitorización de residuos, de una forma sencilla, eficiente y precisa.

Para alcanzar este objetivo general se establecen los siguientes objetivos específicos:

1. Diseñar una arquitectura software que permita la integración de una aplicación móvil, un portal web y un backend centralizado.
2. Desarrollar una aplicación móvil destinada a la captura de observaciones durante campañas de monitorización.
3. Permitir el funcionamiento de la aplicación móvil en condiciones de conectividad limitada mediante mecanismos de almacenamiento local y sincronización diferida.
4. Registrar información geográfica asociada a cada observación utilizando los sensores de posicionamiento disponibles en los dispositivos móviles.

5. Incorporar la gestión de fotografías y metadatos de muestreo como evidencias asociadas a las observaciones registradas.
6. Implementar un portal web que facilite la administración de campañas, usuarios y datos recopilados.
7. Centralizar la información obtenida en una base de datos común que garantice la integridad y consistencia de los datos.
8. Utilizar taxonomías estandarizadas y protocolos de muestreo configurables para favorecer la calidad científica y la comparabilidad de la información registrada.
9. Aplicar metodologías y herramientas de ingeniería del software para garantizar la calidad, mantenibilidad y escalabilidad de la solución desarrollada.

## 1.4. Alcance del proyecto

---

El presente Trabajo Fin de Grado aborda el diseño, desarrollo e implementación de una plataforma software denominada OmniLitter, orientada a facilitar la monitorización y el análisis de la contaminación por residuos en distintos ecosistemas terrestres y acuáticos.

La solución desarrollada está compuesta por una aplicación móvil destinada a la recogida de información en campo, un portal web para la gestión y explotación de los datos recopilados, y un backend centralizado encargado de la lógica de negocio y la persistencia de la información.

Dentro del alcance del proyecto se incluyen las siguientes funcionalidades:

- Diseño e implementación de la arquitectura completa del sistema, incluyendo aplicación móvil, portal web, backend y base de datos.
- Registro de observaciones de residuos utilizando una taxonomía jerárquica estandarizada que garantice la consistencia científica de los datos.
- Captura automática de información geográfica y temporal asociada a cada observación mediante los sensores disponibles en los dispositivos móviles.
- Gestión de fotografías y metadatos asociados a las observaciones registradas.
- Gestión de sesiones de muestreo, incluyendo información contextual como el entorno analizado, el protocolo empleado, la duración de la sesión, el tipo de transecto o cuadrante y las características del área estudiada.
- Funcionamiento en modo offline mediante almacenamiento local de la información y sincronización posterior con el servidor cuando exista conectividad.
- Administración de campañas, usuarios, grupos y permisos mediante un portal web de gestión.

- Visualización de observaciones y campañas mediante mapas interactivos y paneles informativos.
- Exportación de los datos recopilados en formatos adecuados para su posterior explotación y análisis científico.
- Implementación de mecanismos de autenticación, control de acceso y protección de la información almacenada.
- Elaboración de la documentación técnica y académica asociada al desarrollo del sistema.

De esta forma, el alcance del proyecto se centra en proporcionar una plataforma funcional y completa para la recogida, gestión y consulta de datos relacionados con la monitorización de residuos, estableciendo una base tecnológica que permita futuras ampliaciones y desarrollos.

## 2. Contexto y antecedentes

### 2.1. Monitorización de residuos

---

La monitorización de residuos requiere transformar observaciones realizadas en campo en datos comparables, trazables y útiles para el análisis científico. En el caso de las basuras fluviales, la literatura técnica europea señala una carencia relevante: aunque existen diferentes aproximaciones de muestreo, todavía faltan datos sistemáticos y metodologías armonizadas que permitan obtener información cuantitativa comparable entre ríos, periodos temporales y áreas de estudio [3]. Esta limitación dificulta conocer la distribución espacial y temporal de los residuos, identificar fuentes terrestres y evaluar la eficacia de medidas de prevención o mitigación.

El problema no se limita al ámbito fluvial. Los residuos plásticos y otros tipos de basura aparecen en distintos compartimentos ambientales, tanto acuáticos como terrestres. Estudios recientes sobre macroplásticos destacan la necesidad de seleccionar técnicas de observación fiables y coste-efectivas, como censos visuales o muestreos apoyados por drones, y de avanzar hacia protocolos armonizados de monitorización [2]. Esta necesidad es especialmente relevante en entornos terrestres como bosques o zonas urbanas, donde la disponibilidad de datos científicos estructurados y metodologías consolidadas es menor que en playas o riberas.

En este contexto, las acciones de limpieza ciudadana son valiosas como actividad de sensibilización y retirada de residuos, pero sus registros no siempre alcanzan el nivel de detalle, control metodológico y trazabilidad que exige un estudio científico. Para que los datos puedan utilizarse en análisis temporales, comparaciones espaciales o evaluación de políticas públicas, es necesario documentar el método de muestreo, el área cubierta, la frecuencia, la localización y los metadatos asociados a la recogida de datos [3].

### 2.2. Aplicaciones de campo y ciencia ciudadana

---

El uso de aplicaciones digitales permite reducir parte de la brecha entre trabajo de campo, monitorización científica y gestión ambiental. Un precedente directo es el enfoque desarrollado para la observación de macrobasuras flotantes en ríos, en el que una aplicación

de tableta permitió registrar ítems observados, tamaño y geoposición durante sesiones de monitorización, apoyándose en una lista común de elementos y rangos de tamaño para armonizar la recogida y el reporte de datos [4].

Este tipo de herramientas resulta especialmente útil cuando se pretende ampliar la cobertura espacial y temporal de la monitorización. La participación de ONGs, comunidades locales y colaboradores de campo puede multiplicar la capacidad de observación respecto a un equipo científico reducido, siempre que la aplicación guíe la captura de datos mediante protocolos claros, validaciones de entrada, geolocalización, fotografías y metadatos obligatorios. De este modo, la ciencia ciudadana puede generar datos estructurados y reutilizables, no solo registros aislados de limpieza.

## 2.3. Herramientas existentes

---

Las iniciativas existentes para el seguimiento de residuos suelen concentrarse en entornos concretos, como playas, superficie del agua, riberas o campañas de limpieza. Las guías y propuestas recientes de OSPAR para basuras en riberas ponen el foco en producir datos y metadatos comparables, detectar diferencias espaciales y cambios temporales, y evaluar la eficacia de acciones regionales de reducción de basuras [5]. También proponen aspectos operativos como la definición de unidades de muestreo, criterios de selección de sitios, estrategias de muestreo, gestión de datos y control de calidad.

OmniLitter toma estas necesidades como referencia de diseño, pero su objetivo no es sustituir un protocolo científico oficial concreto. La plataforma proporciona una base digital flexible para registrar campañas, sesiones y observaciones en playas, riberas, bosques y zonas urbanas, de forma que el equipo científico pueda aplicar el protocolo adecuado en cada caso y conservar los datos mínimos necesarios para su interpretación posterior.

## 2.4. Taxonomías y datos científicos

---

La clasificación homogénea de residuos es un requisito esencial para comparar resultados entre campañas. El informe técnico del JRC sobre monitorización de basuras fluviales indica que la identificación de macrobasuras debe apoyarse en listas comunes de categorías y destaca la utilidad de la *MSFD Master List of Categories of Litter Items* como base para la comparabilidad entre estudios en ríos y mares adyacentes [3]. Esta lista organiza los ítems por categorías materiales y proporciona un marco común para relacionar objetos observados con posibles usos o fuentes.

Además de la taxonomía, la calidad científica de los datos depende de los metadatos. La monitorización debe registrar información contextual como método de muestreo, compartimento ambiental, tamaño o tipo de ítem, frecuencia, ubicación, condiciones ambientales

y unidades de reporte [3]. En OmniLitter, esta necesidad se traduce en sesiones de muestreo con metadatos asociados, observaciones georreferenciadas, versionado de taxonomía y exportación de datos en formatos reutilizables.

## 2.5. Conclusiones del análisis

---

El análisis del dominio confirma que OmniLitter debe resolver algo más que el registro operativo de residuos. La plataforma debe ayudar a convertir observaciones dispersas en datos armonizados, comparables y georreferenciados, capaces de apoyar estudios científicos y decisiones de gestión. Por ello, los requisitos del sistema se orientan a tres objetivos complementarios: facilitar la recogida de datos en campo, preservar la validez científica mediante protocolos, taxonomías y metadatos, y permitir la explotación posterior de la información para detectar tendencias, identificar fuentes y evaluar medidas de mitigación.

## 3. Descripción del proyecto

### 3.1. Visión general de OmniLitter

---

OmniLitter es una plataforma multiplataforma desarrollada con el objetivo de facilitar la monitorización y el análisis de residuos en distintos tipos de entornos: playas, riberas, bosques y zonas urbanas. El sistema está orientado tanto a investigadores como a colaboradores de campo, proporcionando herramientas que permiten registrar, almacenar y analizar información relacionada con campañas de muestreo de residuos.

La finalidad de la plataforma no es únicamente contabilizar residuos, sino digitalizar y unificar la toma de datos para que las observaciones puedan convertirse en información científica comparable. Para ello, OmniLitter organiza el trabajo en campañas y sesiones de muestreo, registra metadatos de campo, utiliza categorías estandarizadas de residuos y conserva la localización y evidencias asociadas a cada observación.

La arquitectura de OmniLitter se compone de tres elementos principales estrechamente integrados: una API central, una aplicación móvil y un portal web. La aplicación móvil se orienta a la recogida de datos en campo, el portal web a la gestión y explotación de la información, y la API actúa como punto común de autenticación, validación, persistencia y sincronización.

Esta aproximación permite conectar la recogida de datos con la evaluación de tendencias y medidas de mitigación. Al conservar datos cuantitativos, metadatos y trazabilidad por campaña, la plataforma puede apoyar comparaciones antes y después de una intervención, ayudar a identificar fuentes probables de contaminación y aportar evidencias para valorar la eficacia de políticas o acciones de reducción de residuos.

### 3.2. Actores y perfiles de usuario

---

OmniLitter es una plataforma colaborativa en la que distintos usuarios participan en la gestión de campañas de monitorización y en la recogida de observaciones de residuos. Debido a esta naturaleza colaborativa, el sistema implementa un modelo de control de acceso basado en dos conceptos diferenciados: el rol global del usuario dentro de la plataforma y el rol que dicho usuario posee dentro de cada grupo de trabajo.

Esta separación permite que una misma persona pueda desempeñar diferentes responsabilidades según el grupo en el que participe, garantizando al mismo tiempo el aislamiento de la información entre equipos de trabajo independientes.

El acceso a los datos se estructura alrededor de tres elementos fundamentales:

- La cuenta de usuario, que identifica a la persona dentro de la plataforma.
- El grupo de trabajo, que delimita el conjunto de campañas, sesiones y observaciones accesibles.
- El rol asignado, que determina las acciones que el usuario puede realizar sobre dichos datos.

A continuación se describen los principales perfiles contemplados por el sistema.

### **3.2.1. Visitante**

El visitante corresponde a cualquier persona que accede a la parte pública de la plataforma sin haber iniciado sesión. Este perfil únicamente puede consultar información general sobre OmniLitter y acceder a las funcionalidades de autenticación, como el inicio de sesión, el registro de una nueva cuenta o la recuperación de contraseña.

No dispone de acceso a campañas, observaciones, métricas ni información perteneciente a los grupos de trabajo.

### **3.2.2. Usuario autenticado**

El usuario autenticado constituye el perfil base del sistema. Se trata de cualquier persona que dispone de una cuenta válida y ha iniciado sesión correctamente.

Este perfil puede gestionar la información asociada a su propia cuenta, consultar los grupos a los que pertenece, seleccionar un grupo activo y aceptar o rechazar invitaciones recibidas. Sin embargo, los permisos operativos sobre campañas y observaciones dependen del rol que posea dentro de cada grupo.

### **3.2.3. Investigador de campo**

El investigador de campo es el perfil responsable de la recogida de información durante las campañas de monitorización. Su actividad principal se desarrolla mediante la aplicación móvil, desde la cual puede crear sesiones de muestreo, registrar observaciones, clasificar residuos mediante la taxonomía disponible, adjuntar fotografías y capturar la localización geográfica de cada registro.

Además, este perfil puede trabajar en modo offline cuando no existe conectividad, almacenando localmente la información para sincronizarla posteriormente con el servidor.

Aunque puede consultar la información necesaria para desarrollar su trabajo, no dispone de permisos para administrar usuarios, gestionar campañas o modificar la estructura organizativa del grupo.

#### **3.2.4. Administrador de grupo**

El administrador de grupo es el responsable de coordinar la actividad de un grupo de trabajo concreto. Este perfil puede gestionar los miembros del grupo, enviar invitaciones, asignar roles de membresía y administrar las campañas asociadas a dicho grupo.

Entre sus funciones se encuentran la creación, edición y organización de campañas de monitorización, así como la supervisión de la actividad realizada por los investigadores de campo. También dispone de acceso a métricas agregadas e información de seguimiento relacionada con las campañas del grupo.

Sus permisos se limitan exclusivamente a los grupos en los que posee dicho rol. Por tanto, un mismo usuario puede actuar como administrador en un grupo y como investigador de campo en otro diferente.

#### **3.2.5. Superadministrador**

El superadministrador representa el perfil con mayor nivel de autoridad dentro de OmniLitter. Su función principal consiste en administrar la plataforma a nivel global y garantizar su correcto funcionamiento.

Este perfil puede gestionar usuarios, modificar roles globales, supervisar grupos de trabajo y realizar tareas de soporte operativo dentro de la plataforma. A diferencia del administrador de grupo, sus permisos no se encuentran restringidos a un único grupo, sino que abarcan la totalidad del sistema.

Debido a su alcance, este perfil está destinado a tareas de administración y soporte de la plataforma, y no a la realización de actividades habituales de monitorización en campo.

#### **3.2.6. Usuario sin grupo activo**

Finalmente, OmniLitter contempla la situación de usuarios registrados que todavía no pertenecen a ningún grupo o que no han seleccionado uno como grupo activo.

En este estado, el usuario puede gestionar su cuenta personal y revisar invitaciones pendientes, pero no tiene acceso a campañas, observaciones ni funcionalidades relacionadas con la monitorización hasta incorporarse a un grupo de trabajo.

### 3.2.7. Actores de apoyo

Además de los usuarios que interactúan directamente con la plataforma, OmniLitter depende de diversos componentes técnicos que permiten el funcionamiento del sistema.

Entre ellos destacan el servicio de autenticación encargado de validar credenciales y gestionar sesiones, el servicio de correo utilizado para determinadas comunicaciones automáticas, los dispositivos móviles empleados durante las campañas de campo y la infraestructura backend responsable de la lógica de negocio y el almacenamiento persistente de la información.

Aunque estos elementos no representan usuarios propiamente dichos, constituyen actores fundamentales para la correcta ejecución de los procesos definidos por la plataforma.

## 3.3. Plataformas del sistema

---

OmniLitter se concibe como una plataforma compuesta por varios elementos que colaboran entre sí para cubrir las distintas fases del proceso de monitorización de residuos. La naturaleza del problema exige disponer de herramientas adaptadas tanto al trabajo de campo como a la gestión y análisis posterior de la información recopilada.

Para dar respuesta a estas necesidades, el sistema se estructura en tres componentes principales: una aplicación móvil, un portal web y un backend centralizado. Esta separación permite adaptar cada componente a las tareas específicas que debe desempeñar, manteniendo al mismo tiempo una gestión unificada de la información.

### 3.3.1. Aplicación móvil

La aplicación móvil constituye la herramienta principal para la recogida de datos en campo. Está orientada a los usuarios que participan directamente en las campañas de monitorización y les permite registrar observaciones de residuos en el lugar donde se producen.

A través de esta aplicación es posible consultar campañas, iniciar sesiones de muestreo, registrar observaciones, capturar fotografías asociadas a los residuos encontrados y almacenar la localización geográfica de cada registro. Además, la aplicación está diseñada para funcionar en entornos con conectividad limitada, permitiendo continuar el trabajo incluso cuando no existe acceso a Internet y sincronizando posteriormente la información con el servidor.

### 3.3.2. Portal web

El portal web proporciona las herramientas necesarias para la gestión y supervisión de la información recopilada. Esta plataforma está orientada principalmente a administradores de grupo, investigadores y superadministradores.

Desde el portal web es posible gestionar usuarios y grupos de trabajo, crear y administrar campañas de monitorización, consultar observaciones registradas, visualizar información geográfica mediante mapas interactivos y acceder a métricas agregadas sobre los datos recopilados. Asimismo, permite exportar la información para su utilización en procesos posteriores de análisis científico.

A diferencia de la aplicación móvil, cuyo objetivo principal es facilitar la recogida de datos en campo, el portal web está enfocado a la coordinación, revisión y explotación de la información almacenada.

### 3.3.3. Backend y almacenamiento centralizado

El backend actúa como elemento de coordinación entre las distintas plataformas del sistema. Su función consiste en centralizar la lógica de negocio, gestionar la autenticación de usuarios, controlar los permisos de acceso y garantizar la consistencia de los datos almacenados.

Tanto la aplicación móvil como el portal web se comunican con este componente para consultar o registrar información. De esta forma, todas las operaciones se realizan sobre una única fuente de datos común, evitando inconsistencias y facilitando la gestión centralizada del sistema.

La información recopilada durante las campañas se almacena en una base de datos centralizada que actúa como repositorio principal de usuarios, grupos, campañas, sesiones de muestreo, observaciones y fotografías asociadas.

### 3.3.4. Integración de las plataformas

La combinación de estas plataformas permite cubrir el ciclo completo de trabajo de una campaña de monitorización. La aplicación móvil facilita la recogida de datos sobre el terreno, el backend garantiza la gestión y consistencia de la información, y el portal web proporciona las herramientas necesarias para su consulta, supervisión y análisis posterior.

Esta arquitectura permite que cada componente se especialice en una función concreta, favoreciendo la mantenibilidad del sistema y facilitando su evolución futura.

### 3.4. Funcionalidades principales

---

OmniLitter proporciona un conjunto de funcionalidades orientadas a cubrir el ciclo completo de una campaña de monitorización de residuos, desde la organización de los equipos de trabajo hasta la recogida y posterior consulta de los datos obtenidos.

#### 3.4.1. Gestión de usuarios y grupos

La plataforma permite el registro y autenticación de usuarios, así como la organización de estos en grupos de trabajo independientes. Cada grupo dispone de sus propias campañas, sesiones y observaciones, garantizando el aislamiento de la información entre equipos.

Además, el sistema incorpora distintos niveles de permisos que permiten diferenciar entre usuarios de campo, administradores de grupo y administradores globales de la plataforma.

#### 3.4.2. Gestión de campañas

OmniLitter permite crear y administrar campañas de monitorización asociadas a un grupo de trabajo. Cada campaña define el contexto en el que se desarrollarán las actividades de muestreo, incluyendo información descriptiva, fechas de realización y características generales del entorno estudiado.

Las campañas actúan como elemento organizador de toda la información recogida durante el trabajo de campo.

#### 3.4.3. Sesiones de muestreo

Dentro de cada campaña, los usuarios pueden crear sesiones de muestreo que representan actividades concretas de recogida de datos.

Las sesiones permiten registrar información contextual relacionada con el trabajo realizado, como el entorno de estudio, el método de muestreo empleado, el protocolo aplicado, la duración de la actividad, el tipo de transecto o cuadrante y la ubicación geográfica asociada.

Esta funcionalidad aporta trazabilidad a los datos recopilados, permitiendo conocer las condiciones bajo las que se registró cada observación.

#### 3.4.4. Registro de observaciones

El registro de observaciones constituye la funcionalidad principal del sistema. Durante una sesión de muestreo, los usuarios pueden documentar los residuos encontrados indicando su localización, fecha de registro y clasificación correspondiente.

Cada observación queda vinculada a una campaña y una sesión concretas, permitiendo mantener la coherencia y organización de los datos almacenados.

#### **3.4.5. Clasificación mediante taxonomías**

La plataforma incorpora una taxonomía de residuos que permite clasificar las observaciones de forma homogénea y estandarizada, tomando como referencia listas comunes de categorías utilizadas en monitorización de basuras.

El uso de categorías comunes mejora la calidad de los datos recopilados y facilita su comparación entre campañas, grupos de trabajo, entornos y periodos temporales diferentes.

#### **3.4.6. Registro fotográfico y geolocalización**

Las observaciones pueden complementarse mediante fotografías y coordenadas geográficas obtenidas desde el dispositivo móvil.

Estas evidencias permiten contextualizar cada registro, aumentar la fiabilidad de los datos y facilitar posteriores procesos de revisión y análisis.

#### **3.4.7. Trabajo offline y sincronización**

La aplicación móvil ha sido diseñada para utilizarse en entornos donde la conectividad puede ser limitada o inexistente.

Por este motivo, los usuarios pueden continuar registrando observaciones sin conexión a Internet. Cuando la conectividad se restablece, la información almacenada localmente se sincroniza con el sistema central, garantizando la disponibilidad de los datos para el resto de usuarios autorizados.

#### **3.4.8. Consulta y visualización de información**

El portal web permite consultar la información recopilada durante las campañas mediante diferentes mecanismos de visualización.

Los usuarios autorizados pueden revisar observaciones, consultar información geográfica a través de mapas interactivos y acceder a métricas generales relacionadas con la actividad de monitorización realizada.

#### **3.4.9. Exportación de datos**

OmniLitter incorpora mecanismos para exportar la información recopilada en formatos adecuados para su tratamiento posterior.

Esta funcionalidad facilita la reutilización de los datos en herramientas externas de análisis y favorece su utilización en estudios científicos relacionados con la contaminación por residuos, incluyendo la generación de series temporales, comparaciones espaciales y evaluaciones de medidas de mitigación.

### 3.4.10. Resumen funcional

De forma resumida, OmniLitter permite:

- Gestionar usuarios, grupos y permisos.
- Planificar campañas de monitorización.
- Realizar sesiones de muestreo en campo.
- Registrar observaciones geolocalizadas.
- Clasificar residuos mediante una taxonomía estandarizada.
- Asociar fotografías y metadatos a las observaciones.
- Trabajar sin conexión y sincronizar posteriormente los datos.
- Consultar la información recopilada mediante mapas y métricas.
- Exportar los datos para su análisis posterior.

## 3.5. Restricciones y alcance excluido

---

Con el objetivo de acotar el alcance del proyecto y ajustarlo a las limitaciones temporales y de recursos propias de un Trabajo Fin de Grado, se decidió excluir una serie de funcionalidades y actividades que, aunque podrían aportar valor a la plataforma, no resultaban imprescindibles para cumplir los objetivos principales establecidos.

Las principales exclusiones son las siguientes:

- **Soporte multiidioma avanzado.** Aunque la arquitectura de la aplicación permite la internacionalización de la interfaz, el alcance del proyecto se limita al español y al inglés. La incorporación de idiomas adicionales requeriría un esfuerzo significativo de traducción y mantenimiento, además de depender en algunos casos de servicios externos incompatibles con los requisitos de funcionamiento offline de la aplicación móvil.
- **Mantenimiento evolutivo y correctivo posterior a la entrega.** El presente trabajo contempla el análisis, diseño, implementación y validación de la plataforma. Las actividades relacionadas con el mantenimiento posterior, incorporación de nuevas funcionalidades o soporte continuado tras la finalización del proyecto quedan fuera de su alcance.

- **Identificación automática de residuos mediante inteligencia artificial.** Durante las fases iniciales del proyecto se consideró la posibilidad de incorporar mecanismos de reconocimiento automático de residuos a partir de imágenes. Sin embargo, el desarrollo, entrenamiento y validación de modelos de visión artificial requiere una inversión de tiempo, recursos computacionales y conjuntos de datos especializados que exceden las posibilidades de este Trabajo Fin de Grado.
- **Publicación oficial en tiendas de aplicaciones.** Aunque la aplicación móvil ha sido desarrollada para poder ejecutarse en dispositivos reales, la publicación oficial en plataformas como Google Play Store o Apple App Store no forma parte del alcance del proyecto. Este proceso implica requisitos administrativos, costes económicos y procedimientos de revisión que no resultan necesarios para la validación académica de la solución desarrollada.

Estas exclusiones permiten concentrar los esfuerzos de desarrollo en las funcionalidades esenciales de OmniLitter, garantizando una solución completa y funcional dentro de las restricciones temporales y organizativas del proyecto.

## 4. Requisitos y análisis

Este capítulo recoge el proceso seguido para obtener, contrastar y consolidar los requisitos de OmniLitter. Al tratarse de un proyecto desarrollado para un cliente real y dentro de un dominio científico específico, la definición de requisitos no se cerró en una única fase inicial, sino que fue evolucionando a partir de reuniones con el cliente, validación mediante prototipos funcionales y revisión del alcance del sistema.

Por este motivo, los requisitos se presentan en dos niveles. En primer lugar, se describe la evolución del registro de requisitos, indicando qué requisitos estaban presentes en cada sesión de trabajo y qué cambios se introdujeron en cada versión. En segundo lugar, se recoge la versión final consolidada, relacionando cada requisito con el caso de uso al que da soporte cuando procede.

### 4.1. Evolución del registro de requisitos

---

Durante el desarrollo de OmniLitter se manejaron tres versiones principales del registro de requisitos. Cada una representa un momento distinto del proyecto: la toma inicial de requisitos, la validación mediante prototipos y la consolidación posterior a la última reunión de validación. Además, se incorporó un refinamiento de dominio aportado por el cliente, centrado en la necesidad de datos científicos armonizados, comparables y útiles para evaluar medidas de mitigación.

#### 4.1.1. Versión 1: toma inicial de requisitos

**Fecha aproximada:** 09/02/2026.

**Estado:** primera versión del registro de requisitos.

**Objetivo de la sesión:** recoger las necesidades iniciales del cliente y definir una primera lista de requisitos funcionales, no funcionales y científicos.

En la primera reunión se identificó la necesidad de una herramienta capaz de registrar datos sobre residuos en distintos entornos naturales. El cliente planteó la importancia de capturar observaciones geolocalizadas, asociar fotografías, registrar metadatos del muestreo, trabajar sin conexión y sincronizar posteriormente los datos con un servidor. También

se identificaron necesidades de usuarios, grupos, roles, visualización en mapa, estadísticas y exportación.

**Cuadro 4.1:** Requisitos identificados en la versión 1.

| ID     | Descripción                                                                | Prioridad |
|--------|----------------------------------------------------------------------------|-----------|
| RF-01  | Registro de tipo de residuo a partir de tipos preestablecidos.             | Crítica   |
| RF-02  | Registro del tamaño del residuo observado.                                 | Media     |
| RF-03  | Geolocalización automática de cada observación mediante GPS.               | Crítica   |
| RF-04  | Registro automático de fecha y hora del avistamiento.                      | Alta      |
| RF-05  | Asociación de una o varias fotografías al registro de residuo.             | Alta      |
| RF-06  | Funcionamiento sin conexión, con almacenamiento local de datos.            | Crítica   |
| RF-07  | Sincronización de datos locales con el servidor al recuperar conectividad. | Crítica   |
| RF-08  | Selección del tipo de entorno antes de iniciar el muestreo.                | Alta      |
| RF-09  | Registro de parámetros de muestreo, como ancho de observación o duración.  | Alta      |
| RF-10  | Interfaz disponible en español e inglés.                                   | Media     |
| RF-11  | Creación de cuentas de usuario.                                            | Alta      |
| RF-12  | Creación de grupos de trabajo y asociación de usuarios.                    | Crítica   |
| RF-13  | Soporte de roles de usuario.                                               | Crítica   |
| RF-14  | Visualización de observaciones geolocalizadas en mapa.                     | Alta      |
| RF-15  | Cálculo de estadísticas básicas a partir de los datos almacenados.         | Media     |
| RF-16  | Privacidad por grupo para limitar la visibilidad de datos.                 | Crítica   |
| RNF-01 | Interfaz optimizada para tablet y uso en campo.                            | Alta      |
| RNF-02 | Comunicación segura entre aplicación móvil y servidor mediante HTTPS.      | Crítica   |
| RNF-03 | Compatibilidad de la aplicación móvil con Android e iOS.                   | Media     |
| RDC-01 | Almacenamiento de datos según la taxonomía proporcionada por el cliente.   | Crítica   |
| RDC-02 | Exportación de datos en formato CSV para análisis científico posterior.    | Alta      |

Esta versión constituye la línea base del proyecto y permite identificar los bloques iniciales: captura de datos, geolocalización, fotografías, funcionamiento offline, sincronización, usuarios, grupos, roles, mapa, estadísticas, seguridad, compatibilidad, taxonomía y exportación.

#### 4.1.2. Versión 2: validación mediante prototipos

**Estado:** versión de trabajo posterior a la toma inicial.

**Objetivo de la sesión:** contrastar los requisitos iniciales con prototipos funcionales del portal web y de la aplicación móvil.

Tras la primera toma de requisitos, se elaboraron prototipos que representaban los principales flujos del sistema: autenticación, selección de grupo activo, creación de sesiones de muestreo, registro de observaciones, consulta de datos, mapa y sincronización. Esta revisión permitió transformar necesidades generales de la V1 en requisitos más cercanos al funcionamiento real del sistema.

**Cuadro 4.2:** Requisitos definidos en la versión 2 tras la validación con prototipos.

| ID      | Descripción                                                                                                             | Prioridad |
|---------|-------------------------------------------------------------------------------------------------------------------------|-----------|
| RF-001  | Inicio de sesión en portal web y aplicación móvil.                                                                      | Crítica   |
| RF-002  | Cierre de sesión y revocación de sesión local.                                                                          | Alta      |
| RF-004  | Roles y control de acceso por grupo.                                                                                    | Crítica   |
| RF-005  | Creación de grupos y asociación de usuarios.                                                                            | Crítica   |
| RF-006  | Selección de grupo activo para filtrar datos y acciones.                                                                | Alta      |
| RF-007  | Trabajo por sesiones de muestreo en la aplicación móvil.                                                                | Crítica   |
| RF-008  | Captura de coordenadas GPS en sesiones y observaciones.                                                                 | Crítica   |
| RF-009  | Registro de metadatos de muestreo asociados a la sesión, incluyendo protocolo, entorno y parámetros del área observada. | Alta      |
| RF-010  | Fotografías asociadas a observaciones.                                                                                  | Alta      |
| RF-012  | Navegación por taxonomía jerárquica basada en categorías estandarizadas de residuos.                                    | Crítica   |
| RF-015  | Registro de cantidad observada como entero positivo.                                                                    | Crítica   |
| RF-020  | Revisión de sesión antes de finalizarla.                                                                                | Alta      |
| RF-022  | Funcionamiento sin conexión durante la captura de campo.                                                                | Crítica   |
| RF-023  | Cola de sincronización y subida posterior de datos pendientes.                                                          | Crítica   |
| RF-027  | Listado de sesiones y observaciones con filtros en el portal.                                                           | Alta      |
| RF-028  | Visualización de observaciones en mapa.                                                                                 | Alta      |
| RF-030  | Dashboard con métricas agregadas.                                                                                       | Media     |
| RF-031  | Exportación de datos para análisis externo.                                                                             | Media     |
| RF-032  | Gestión de usuarios desde el portal.                                                                                    | Alta      |
| RF-033  | Gestión de grupos y membresías desde el portal.                                                                         | Alta      |
| RF-035  | Soporte de interfaz en español e inglés.                                                                                | Media     |
| RNF-001 | Seguridad en comunicaciones y acceso autenticado.                                                                       | Crítica   |
| RNF-002 | Aislamiento de datos entre grupos de investigación.                                                                     | Crítica   |

| ID      | Descripción                                                            | Prioridad |
|---------|------------------------------------------------------------------------|-----------|
| RNF-003 | Enfoque <i>offline-first</i> para la aplicación móvil.                 | Crítica   |
| RNF-004 | Fiabilidad de sincronización ante cortes de red o cierres inesperados. | Crítica   |
| RNF-005 | Usabilidad orientada al trabajo de campo.                              | Alta      |
| RNF-008 | Trazabilidad entre requisitos, operaciones relevantes y pruebas.       | Alta      |
| RDC-001 | Taxonomía jerárquica con código identificador estable.                 | Crítica   |
| RDC-002 | Metadatos científicos de sesión.                                       | Crítica   |
| RDC-003 | Metadatos técnicos de fotografías cuando estén disponibles.            | Media     |
| RDC-004 | Exportación científica con campos suficientes para análisis.           | Media     |
| RDC-005 | Versión de taxonomía asociada a sesiones y observaciones.              | Alta      |

Respecto a la V1, la creación de cuentas pasa a ser un flujo de autenticación completo, los roles se refinan como control de acceso por grupo, los metadatos se organizan alrededor de sesiones de muestreo y la taxonomía deja de ser una lista simple para convertirse en una estructura jerárquica navegable.

#### 4.1.3. Versión 3: consolidación tras la última reunión y revisión de dominio

**Fecha aproximada:** 10/06/2026.

**Estado:** versión final consolidada de requisitos.

**Objetivo de la sesión:** revisar el flujo real de la aplicación móvil, especialmente sesiones, sincronización, mapa y operación posterior del sistema, e incorporar la revisión científica posterior aportada por el cliente.

En la última reunión se revisó el funcionamiento de la aplicación móvil en relación con las sesiones de muestreo. Se reforzó la necesidad de consultar qué sesiones estaban pendientes, cuántas observaciones contenían, si una sesión seguía activa y cómo debía finalizarse antes de sincronizar. También se aclaró que la memoria debía documentar el despliegue y la administración básica, dejando el mantenimiento evolutivo posterior fuera del alcance académico del proyecto.

La información aportada por el cliente reforzó la motivación científica del sistema: la falta de datos espaciales y temporales, la ausencia de metodologías armonizadas en algunos entornos terrestres y la necesidad de capturar datos y metadatos de campo con suficiente calidad para comparar campañas, detectar tendencias y evaluar la eficacia de políticas o medidas de mitigación. Esta revisión se alinea con la literatura sobre basuras fluviales, que reclama datos comparables, listas comunes de categorías, metadatos y compatibilidad entre evaluaciones continentales, costeras y marinas [3, 4, 5].

**Cuadro 4.3:** Requisitos consolidados o refinados en la versión 3.

| ID      | Descripción                                                                                     | Prioridad |
|---------|-------------------------------------------------------------------------------------------------|-----------|
| RF-021  | Estado de sincronización de sesiones: sincronizada, pendiente, activa o con error.              | Alta      |
| RF-022  | Funcionamiento offline completo con sesiones, observaciones, fotografías y coordenadas locales. | Crítica   |
| RF-023  | Sincronización automática o manual al recuperar conectividad, sin pérdida de datos.             | Crítica   |
| RF-024  | Finalización de sesiones activas antes de sincronizarlas cuando sea necesario.                  | Alta      |
| RF-025  | Pantalla de sincronización con sesiones pendientes, sincronizadas y número de observaciones.    | Alta      |
| RF-028  | Visualización en mapa con filtros por entorno, campaña, sesión o localización cuando proceda.   | Alta      |
| RF-035  | Interfaz en español e inglés, sin soporte multiidioma avanzado dentro del alcance.              | Media     |
| RF-036  | Documentación técnica de instalación, despliegue y administración básica.                       | Alta      |
| RF-037  | Documentación del mantenimiento posterior como actividad fuera del alcance del TFG.             | Media     |
| RNF-006 | Operabilidad mediante documentación suficiente para despliegue y administración básica.         | Alta      |
| RNF-007 | Mantenibilidad orientada a facilitar futuras modificaciones de categorías o metadatos.          | Media     |
| RDC-006 | Versionado y documentación de futuras evoluciones de categorías, taxonomías y metadatos.        | Media     |

La V3 no sustituye al conjunto completo de la V2, sino que lo consolida. En particular, precisa el flujo *offline-first*, refuerza la trazabilidad científica, explicita el estado de sincronización de las sesiones y acota el alcance operativo posterior a la entrega.

## 4.2. Casos de uso de referencia

Los casos de uso de referencia permiten relacionar los requisitos finales con interacciones concretas del sistema. No se pretende recoger todos los escenarios posibles, sino los flujos principales necesarios para validar el comportamiento de OmniLitter.

- **CU-01 – Autenticarse en el sistema.** El usuario accede al portal web o a la aplicación móvil mediante credenciales válidas.
- **CU-02 – Seleccionar grupo activo.** El usuario autenticado elige el grupo de

trabajo sobre el que se filtrarán campañas, sesiones y observaciones.

- **CU-03 – Crear sesión de muestreo.** El investigador de campo inicia una sesión, indicando campaña, entorno y metadatos de muestreo.
- **CU-04 – Registrar observación.** El investigador registra un residuo indicando taxonomía, cantidad, GPS, fecha, comentarios y fotografía cuando proceda.
- **CU-05 – Revisar y finalizar sesión.** El usuario comprueba las observaciones registradas en una sesión activa y la finaliza antes de sincronizarla.
- **CU-06 – Sincronizar datos pendientes.** El usuario o el sistema sube a la base de datos principal las sesiones y observaciones pendientes cuando existe conexión.
- **CU-07 – Consultar sesiones y observaciones.** El usuario autorizado revisa desde el portal web sesiones, observaciones, filtros, mapas y métricas agregadas.
- **CU-08 – Administrar usuarios y grupos.** Un administrador gestiona usuarios, grupos, membresías y roles dentro del alcance implementado.
- **CU-09 – Exportar datos.** Un usuario autorizado exporta los datos recopilados para su análisis posterior en herramientas externas.
- **CU-10 – Desplegar y operar el sistema.** Una persona con perfil técnico utiliza la documentación del proyecto para instalar, desplegar y administrar la plataforma de forma básica.

### 4.3. Requisitos finales consolidados

---

En esta sección se presenta la versión final consolidada de requisitos. Para cada bloque se introduce brevemente el caso de uso correspondiente y, a continuación, se describen los requisitos asociados mediante tarjetas.

#### 4.3.1. CU-01 – Autenticarse en el sistema

Este caso de uso permite que un usuario acceda al sistema desde el portal web o desde la aplicación móvil. Es la base para aplicar permisos, acceder a grupos de trabajo y registrar actividad asociada a una cuenta concreta.

**RF-001 – Inicio de sesión en portal web y aplicación móvil**

**Tipo:** funcional.

**Prioridad:** crítica.

**Caso de uso asociado:** CU-01.

**Descripción:** el sistema deberá permitir el inicio de sesión de usuarios en el portal web y en la aplicación móvil.

**Criterio de aceptación:** un usuario con credenciales válidas puede acceder al sistema desde ambas plataformas y visualizar las funcionalidades correspondientes a su perfil.

**RF-002 – Cierre de sesión y revocación local**

**Tipo:** funcional.

**Prioridad:** alta.

**Caso de uso asociado:** CU-01.

**Descripción:** el sistema deberá permitir el cierre de sesión y la revocación de la sesión local.

**Criterio de aceptación:** el usuario puede cerrar sesión y no puede acceder a zonas protegidas sin autenticarse de nuevo.

**4.3.2. CU-02 – Seleccionar grupo activo**

Este caso de uso permite que un usuario que pertenece a uno o varios grupos pueda trabajar sobre un contexto concreto. El grupo activo determina qué campañas, sesiones, observaciones, métricas y acciones están disponibles.

**RF-004 – Roles y control de acceso por grupo**

**Tipo:** funcional.

**Prioridad:** crítica.

**Caso de uso asociado:** CU-02, CU-07 y CU-08.

**Descripción:** el sistema deberá soportar roles y control de acceso por grupo, garantizando que cada usuario solo pueda acceder a las funcionalidades y datos permitidos dentro del grupo correspondiente.

**Criterio de aceptación:** las funcionalidades visibles y las operaciones permitidas varían según el rol del usuario dentro del grupo activo.

**RF-006 – Selección de grupo activo**

**Tipo:** funcional.

**Prioridad:** alta.

**Caso de uso asociado:** CU-02.

**Descripción:** el sistema deberá permitir seleccionar un grupo activo para filtrar datos y acciones.

**Criterio de aceptación:** las campañas, sesiones, observaciones y métricas mostradas se corresponden con el grupo activo seleccionado.

**4.3.3. CU-03 – Crear sesión de muestreo**

Este caso de uso representa el inicio del trabajo de campo. Antes de registrar observaciones individuales, el investigador crea una sesión de muestreo que almacena el contexto de la actividad, como campaña, entorno, parámetros de muestreo, ubicación inicial y metadatos asociados.

**RF-007 – Trabajo por sesiones de muestreo**

**Tipo:** funcional.

**Prioridad:** crítica.

**Caso de uso asociado:** CU-03.

**Descripción:** la aplicación móvil deberá organizar el trabajo de campo mediante sesiones de muestreo.

**Criterio de aceptación:** el usuario puede crear una sesión y asociar a ella las observaciones registradas durante el muestreo.

**RF-008 – Captura de coordenadas GPS**

**Tipo:** funcional.

**Prioridad:** crítica.

**Caso de uso asociado:** CU-03 y CU-04.

**Descripción:** la aplicación móvil deberá capturar coordenadas GPS al iniciar sesiones de muestreo y al registrar observaciones.

**Criterio de aceptación:** las sesiones y observaciones almacenan latitud y longitud cuando el dispositivo dispone de permisos y señal GPS suficiente.

**RF-009 – Metadatos de muestreo**

**Tipo:** funcional.

**Prioridad:** alta.

**Caso de uso asociado:** CU-03.

**Descripción:** el sistema deberá permitir registrar metadatos de muestreo, como entorno, protocolo utilizado, tipo de método, parámetros del transecto o cuadrante, duración, área observada y cualquier información necesaria para interpretar el dato en su contexto científico.

**Criterio de aceptación:** la sesión queda asociada a los metadatos necesarios para interpretar correctamente los datos recopilados y comparar campañas realizadas en distintos entornos o periodos.

**4.3.4. CU-04 – Registrar observación**

Este caso de uso constituye el flujo principal de la aplicación móvil. Durante una sesión de muestreo, el investigador registra un residuo observado, seleccionando su tipo dentro de la taxonomía, indicando cantidad y asociando información de ubicación, fecha, comentarios y fotografías cuando proceda.

**RF-010 – Fotografías asociadas a observaciones**

**Tipo:** funcional.

**Prioridad:** alta.

**Caso de uso asociado:** CU-04.

**Descripción:** la aplicación móvil deberá permitir adjuntar fotografías a una observación.

**Criterio de aceptación:** el usuario puede capturar o seleccionar una fotografía y esta queda vinculada a la observación correspondiente.

**RF-012 – Navegación por taxonomía jerárquica**

**Tipo:** funcional.

**Prioridad:** crítica.

**Caso de uso asociado:** CU-04.

**Descripción:** la aplicación móvil deberá permitir navegar por una taxonomía jerárquica de residuos basada en categorías estandarizadas, tomando como referencia listas comunes utilizadas en monitorización de basuras.

**Criterio de aceptación:** el usuario puede seleccionar un tipo de residuo desde una estructura organizada por categorías y subcategorías, conservando un código estable asociado al ítem seleccionado.

**RF-015 – Cantidad observada**

**Tipo:** funcional.

**Prioridad:** crítica.

**Caso de uso asociado:** CU-04.

**Descripción:** el sistema deberá registrar la cantidad observada como un valor entero positivo.

**Criterio de aceptación:** no se permite guardar una observación con cantidad vacía, negativa o igual a cero.

**4.3.5. CU-05 – Revisar y finalizar sesión**

Este caso de uso permite revisar la información registrada antes de cerrar una sesión de muestreo. La revisión es importante para detectar errores antes de sincronizar los datos con el sistema central.

**RF-020 – Revisión de sesión antes de finalizarla**

**Tipo:** funcional.

**Prioridad:** alta.

**Caso de uso asociado:** CU-05.

**Descripción:** la aplicación móvil deberá permitir revisar una sesión antes de finalizarla, comprobando las observaciones registradas antes de su sincronización o envío al servidor.

**Criterio de aceptación:** el usuario puede consultar las observaciones de una sesión activa y comprobar su contenido antes de cerrarla.

**RF-024 – Finalización de sesiones activas**

**Tipo:** funcional.

**Prioridad:** alta.

**Caso de uso asociado:** CU-05 y CU-06.

**Descripción:** la aplicación móvil deberá permitir finalizar sesiones activas antes de sincronizarlas cuando sea necesario.

**Criterio de aceptación:** una sesión activa puede cerrarse desde el flujo de revisión o sincronización antes de enviarse al servidor.

**4.3.6. CU-06 – Sincronizar datos pendientes**

Este caso de uso cubre el funcionamiento offline y la sincronización posterior de los datos. Permite que las sesiones y observaciones registradas sin conexión se mantengan almacenadas localmente hasta que puedan enviarse al servidor.

**RF-021 – Estado de sincronización de sesiones**

**Tipo:** funcional.

**Prioridad:** alta.

**Caso de uso asociado:** CU-06.

**Descripción:** la aplicación móvil deberá mostrar el estado de sincronización de cada sesión, diferenciando sesiones sincronizadas, pendientes, activas o con error.

**Criterio de aceptación:** el usuario puede identificar claramente el estado de cada sesión desde la interfaz de sincronización.

**RF-022 – Funcionamiento offline completo**

**Tipo:** funcional.

**Prioridad:** crítica.

**Caso de uso asociado:** CU-03, CU-04 y CU-06.

**Descripción:** la aplicación móvil deberá permitir la captura sin conexión durante el trabajo de campo, almacenando localmente sesiones, observaciones, fotografías y coordenadas GPS.

**Criterio de aceptación:** el usuario puede crear sesiones y observaciones sin conexión, quedando los datos disponibles para sincronización posterior.

**RF-023 – Sincronización automática o manual**

**Tipo:** funcional.

**Prioridad:** crítica.

**Caso de uso asociado:** CU-06.

**Descripción:** el sistema deberá disponer de una cola de sincronización y permitir la subida de datos pendientes al recuperar conectividad, incluyendo sincronización automática o manual sin pérdida de datos.

**Criterio de aceptación:** los registros pendientes se envían al servidor cuando existe conexión y permanecen disponibles si se produce un error.

**RF-025 – Pantalla de sincronización**

**Tipo:** funcional.

**Prioridad:** alta.

**Caso de uso asociado:** CU-06.

**Descripción:** la aplicación móvil deberá incluir una pantalla de sincronización con detalle de sesiones pendientes y número de ítems u observaciones asociados.

**Criterio de aceptación:** el usuario puede consultar qué sesiones están pendientes, cuáles están sincronizadas y cuántas observaciones contiene cada una.

**4.3.7. CU-07 – Consultar sesiones y observaciones**

Este caso de uso representa la explotación de la información desde el portal web. Los usuarios autorizados pueden consultar sesiones, observaciones, mapas y métricas agregadas,

siempre respetando el aislamiento por grupo.

#### RF-027 – Listados de sesiones y observaciones con filtros

**Tipo:** funcional.

**Prioridad:** alta.

**Caso de uso asociado:** CU-07.

**Descripción:** el portal web deberá permitir consultar sesiones y observaciones mediante listados y filtros.

**Criterio de aceptación:** un usuario autorizado puede filtrar datos por grupo, campaña, sesión, entorno u otros criterios disponibles.

#### RF-028 – Visualización en mapa con filtros

**Tipo:** funcional.

**Prioridad:** alta.

**Caso de uso asociado:** CU-07.

**Descripción:** el portal web deberá permitir visualizar observaciones en un mapa o vista geográfica, con filtros por entorno, campaña, sesión o localización cuando proceda.

**Criterio de aceptación:** las observaciones geolocalizadas pueden consultarse espacialmente y filtrarse según los criterios definidos.

#### RF-030 – Dashboard con métricas agregadas

**Tipo:** funcional.

**Prioridad:** media.

**Caso de uso asociado:** CU-07.

**Descripción:** el portal web deberá mostrar un dashboard con métricas agregadas.

**Criterio de aceptación:** el usuario puede visualizar indicadores, gráficos o resúmenes calculados a partir de las observaciones almacenadas.

### 4.3.8. CU-08 – Administrar usuarios y grupos

Este caso de uso permite a los usuarios autorizados administrar la estructura organizativa de la plataforma, incluyendo usuarios, grupos, membresías y roles.

#### RF-005 – Creación de grupos y asociación de usuarios

**Tipo:** funcional.

**Prioridad:** crítica.

**Caso de uso asociado:** CU-08.

**Descripción:** el sistema deberá permitir la creación de grupos y la asociación de usuarios a dichos grupos.

**Criterio de aceptación:** un usuario con permisos suficientes puede crear un grupo y añadir o eliminar miembros.

**RF-032 – Gestión de usuarios desde el portal**

**Tipo:** funcional.

**Prioridad:** alta.

**Caso de uso asociado:** CU-08.

**Descripción:** el portal web deberá permitir la gestión de usuarios por parte de perfiles autorizados.

**Criterio de aceptación:** un usuario autorizado puede consultar, crear, editar o desactivar usuarios según los permisos definidos.

**RF-033 – Gestión de grupos y membresías**

**Tipo:** funcional.

**Prioridad:** alta.

**Caso de uso asociado:** CU-08.

**Descripción:** el portal web deberá permitir la gestión de grupos y membresías.

**Criterio de aceptación:** un administrador puede gestionar miembros, roles y asociaciones entre usuarios y grupos.

**4.3.9. CU-09 – Exportar datos**

Este caso de uso permite reutilizar la información registrada en OmniLitter para análisis externo. La exportación resulta importante para estudios científicos, informes y tratamiento posterior de los datos.

**RF-031 – Exportación de datos**

**Tipo:** funcional.

**Prioridad:** media.

**Caso de uso asociado:** CU-09.

**Descripción:** el sistema deberá permitir exportar datos en un formato adecuado para análisis posterior, preferentemente CSV.

**Criterio de aceptación:** un usuario autorizado puede obtener un fichero exportable con los registros seleccionados.

**4.3.10. CU-10 – Desplegar y operar el sistema**

Este caso de uso no representa una interacción habitual de los usuarios finales, sino una necesidad operativa identificada tras la última reunión. El cliente planteó dudas sobre cómo mantener o modificar el sistema en el futuro, por lo que se decidió documentar el despliegue y la administración básica, dejando el mantenimiento evolutivo fuera del alcance del TFG.

**RF-036 – Documentación técnica de instalación y despliegue**

**Tipo:** funcional de soporte.

**Prioridad:** alta.

**Caso de uso asociado:** CU-10.

**Descripción:** el proyecto deberá incluir documentación técnica de instalación, despliegue y administración básica del sistema.

**Criterio de aceptación:** una persona con perfil técnico dispone de instrucciones suficientes para desplegar, configurar y administrar de forma básica la plataforma.

**RF-037 – Mantenimiento posterior fuera del alcance**

**Tipo:** documentación de alcance.

**Prioridad:** media.

**Caso de uso asociado:** CU-10 y trabajo futuro.

**Descripción:** el proyecto deberá documentar que los cambios futuros de categorías, taxonomías o metadatos requieren mantenimiento técnico posterior y quedan fuera del alcance de desarrollo del TFG.

**Criterio de aceptación:** la memoria identifica explícitamente estas actividades como limitaciones de alcance o líneas de trabajo futuro.

## 4.4. Requisitos no funcionales consolidados

Además de los requisitos funcionales, OmniLitter debe cumplir una serie de condiciones de calidad que afectan al sistema en su conjunto.

**RNF-001 – Seguridad**

**Prioridad:** crítica.

**Descripción:** la comunicación entre clientes y backend deberá realizarse mediante HTTPS/TLS en entornos de despliegue, y el acceso a datos deberá estar protegido mediante autenticación y autorización por grupo.

**RNF-002 – Aislamiento de datos**

**Prioridad:** crítica.

**Descripción:** los usuarios solo podrán acceder a datos de los grupos a los que pertenecen. Esta separación deberá aplicarse en listados, mapas, métricas, campañas y observaciones.

**RNF-003 – Offline-first**

**Prioridad:** crítica.

**Descripción:** las operaciones principales de captura en campo no deberán depender de conectividad.

**RNF-004 – Fiabilidad de sincronización**

**Prioridad:** crítica.

**Descripción:** la sincronización deberá ser reintentable y no deberá perder datos ante cortes de red, cierres inesperados de la aplicación o sesiones pendientes.

**RNF-005 – Usabilidad en campo**

**Prioridad:** alta.

**Descripción:** la aplicación móvil deberá facilitar capturas rápidas, revisión de sesiones y comprensión clara del estado pendiente, activo, sincronizado o con error.

**RNF-006 – Operabilidad**

**Prioridad:** alta.

**Descripción:** el sistema deberá acompañarse de documentación suficiente para que una persona con perfil técnico pueda instalar, desplegar, configurar y administrar la plataforma de forma básica.

**RNF-007 – Mantenibilidad**

**Prioridad:** media.

**Descripción:** el diseño deberá facilitar futuras modificaciones de categorías, taxonomías o metadatos, aunque dichas modificaciones quedan fuera del alcance de mantenimiento incluido en el TFG.

**RNF-008 – Trazabilidad**

**Prioridad:** alta.

**Descripción:** el sistema y la memoria deberán mantener relación entre requisitos, casos de uso, decisiones de diseño y pruebas, especialmente en sincronización, permisos y datos científicos.

## 4.5. Requisitos de datos científicos consolidados

Al tratarse de una herramienta para captura y análisis de datos científicos, OmniLitter debe preservar suficiente información contextual para que las observaciones puedan interpretarse y compararse posteriormente.

**RDC-001 – Taxonomía jerárquica con códigos estables**

**Prioridad:** crítica.

**Descripción:** la taxonomía de residuos deberá presentarse de forma jerárquica y conservar el código identificador de cada tipo de residuo, de acuerdo con la necesidad de utilizar listas comunes de categorías para favorecer la comparabilidad entre estudios [3].

**RDC-002 – Metadatos de muestreo**

**Prioridad:** crítica.

**Descripción:** las sesiones deberán almacenar metadatos de muestreo suficientes para interpretar los datos recopilados, incluyendo entorno, protocolo, método de muestreo, unidad o área observada, fecha, duración y localización cuando proceda.

**RDC-003 – Metadatos de fotografías**

**Prioridad:** media.

**Descripción:** las fotografías deberán conservar metadatos relevantes cuando estén disponibles, como fecha de captura, dimensiones, formato o coordenadas EXIF.

**RDC-004 – Exportación científica**

**Prioridad:** media.

**Descripción:** los datos exportados deberán incluir campos suficientes para análisis posterior y, cuando sea posible, un diccionario de columnas, unidades, metadatos de muestreo y versión de taxonomía utilizada.

**RDC-005 – Versionado de taxonomía**

**Prioridad:** alta.

**Descripción:** la versión de taxonomía utilizada deberá quedar asociada a sesiones y observaciones para garantizar reproducibilidad.

**RDC-006 – Evolución futura de categorías y metadatos**

**Prioridad:** media.

**Descripción:** las futuras modificaciones de categorías, taxonomías o metadatos deberán documentarse y versionarse para no comprometer la comparabilidad de los datos históricos.

## 4.6. Trazabilidad final

La trazabilidad permite justificar el origen de los requisitos consolidados y relacionar cada decisión con las fuentes empleadas durante el proyecto.

### Trazabilidad desde la V1

El registro inicial de requisitos permitió identificar las necesidades base del sistema: registro de residuos, GPS, fotografías, fecha y hora, funcionamiento offline, sincronización, usuarios, grupos, roles, mapa, estadísticas, privacidad, seguridad, compatibilidad, taxonomía y exportación. Estos requisitos se transformaron posteriormente en los bloques funcionales de captura de campo, gestión de usuarios y grupos, consulta en portal, sincronización y requisitos científicos.

## Trazabilidad desde la V2

La validación mediante prototipos permitió refinar los requisitos iniciales y organizarlos según los flujos reales del sistema. De esta sesión derivan la selección de grupo activo, la organización por sesiones de muestreo, la revisión de sesión, la cola de sincronización, los listados con filtros, el dashboard y la separación formal entre requisitos funcionales, no funcionales y de datos científicos.

## Trazabilidad desde la V3

La última reunión permitió consolidar el flujo offline y de sincronización. De esta sesión derivan de forma explícita el estado de sincronización de sesiones, la pantalla de sincronización, la finalización de sesiones activas antes de sincronizar, el refinamiento de la vista de mapa con filtros y la aclaración de la documentación técnica y del mantenimiento posterior fuera del alcance del TFG.

## Relación entre fuentes, cambios y requisitos finales

- La necesidad inicial de registrar residuos se consolida en RF-007, RF-008, RF-010, RF-012, RF-015, RDC-001 y RDC-002.
- La necesidad inicial de trabajar sin conexión se consolida en RF-021, RF-022, RF-023, RF-024, RF-025, RNF-003 y RNF-004.
- La necesidad inicial de gestionar usuarios, grupos y privacidad se consolida en RF-001, RF-002, RF-004, RF-005, RF-006, RF-032, RF-033, RNF-001 y RNF-002.
- La necesidad inicial de consultar y analizar datos se consolida en RF-027, RF-028, RF-030, RF-031 y RDC-004.
- La necesidad de preservar valor científico se consolida en RDC-001, RDC-002, RDC-003, RDC-004, RDC-005 y RDC-006.
- La revisión científica aportada por el cliente y las fuentes consultadas refuerzan RF-009, RF-012, RF-031, RDC-001, RDC-002, RDC-004 y RDC-005, al vincular la aplicación con la toma de datos armonizada, el uso de listas comunes de categorías, la captura de metadatos y la generación de datos comparables para evaluar tendencias y medidas de mitigación.
- Las dudas planteadas en la última reunión sobre despliegue y mantenimiento se consolidan en RF-036, RF-037, RNF-006 y RNF-007.

En conclusión, la versión final de requisitos mantiene el objetivo principal de OmniLitter: desarrollar una plataforma formada por aplicación móvil, portal web y backend centralizado para la captura, gestión y análisis de datos de residuos. La evolución respecto a

las versiones iniciales se centra en precisar el flujo offline y de sincronización, reforzar el estado de las sesiones, completar la trazabilidad científica y aclarar el alcance operativo posterior a la entrega.

## 5. Planificación y gestión

El desarrollo de OmniLitter se ha planteado de forma iterativa, combinando tareas de análisis, diseño, prototipado e implementación. Esta aproximación resulta adecuada para un proyecto con cliente real, ya que permite validar decisiones antes de invertir esfuerzo en una implementación definitiva.

### 5.1. Metodología de trabajo

---

La metodología seguida se basa en ciclos cortos de trabajo:

1. Recogida de necesidades con el cliente y tutores.
2. Definición de requisitos funcionales, no funcionales y de datos científicos.
3. Elaboración de prototipos funcionales para validar flujos de usuario.
4. Revisión con el cliente y refinamiento del prototipo.
5. Diseño de la arquitectura, base de datos y componentes del sistema.
6. Implementación progresiva de las partes finales del proyecto.

Aunque no se ha aplicado una metodología ágil formal completa, el proyecto adopta una forma de trabajo incremental: cada avance genera un artefacto revisable (documento de requisitos, prototipo, diagrama, módulo de código o capítulo de memoria) que se contrasta antes de continuar.

### 5.2. Herramientas de gestión

---

Para trasladar esta aproximación a la ejecución diaria del proyecto se ha utilizado un backlog funcional agrupado por sprints. Cada elemento del backlog representa una capacidad observable del sistema y se ha ordenado según sus dependencias técnicas y funcionales. De este modo, las tareas iniciales no se han tratado como actividades aisladas, sino como una secuencia de construcción progresiva: primero se establece la infraestructura común de autenticación y permisos, después se consolida la navegación de los clientes y, finalmente, se habilitan los primeros flujos de captura y persistencia de datos.

### 5.3. Backlog y organización por sprints

La planificación del desarrollo se ha estructurado mediante un backlog funcional priorizado. Este backlog recoge las funcionalidades necesarias para transformar los requisitos validados en la Sección 4 en incrementos implementables, manteniendo la coherencia con la arquitectura de tres capas definida para OmniLitter: portal web, aplicación móvil y backend REST con persistencia relacional.

#### 5.3.0.1. Sprint 1: Infraestructura, autenticación y flujo básico de captura

El primer bloque de trabajo corresponde al Sprint 1 y concentra las tareas T01 a T10. Estas tareas constituyen la base inicial del sistema, ya que introducen la configuración funcional común, la autenticación, el control de acceso por grupos, la navegación del portal web, la administración inicial, el modelo de sesiones de muestreo, la taxonomía y el flujo básico de creación de observaciones desde la aplicación móvil. Su selección responde a una decisión de planificación: antes de incorporar módulos analíticos, exportaciones avanzadas o sincronización completa, era necesario disponer de una estructura mínima que conectara usuarios, grupos, sesiones, observaciones y clientes.

**Cuadro 5.1:** Resumen del backlog inicial integrado en el Sprint 1.

| ID  | Funcionalidad                                                | Área                       | Dependencias  |
|-----|--------------------------------------------------------------|----------------------------|---------------|
| T01 | Login y logout común en app y portal                         | Backend, web y app         | Ninguna       |
| T02 | Roles, permisos y grupo activo                               | Backend y web              | T01           |
| T03 | Portal web: layout, navegación y rutas protegidas            | Web                        | T01, T02      |
| T04 | Portal admin: usuarios, grupos y membresías                  | Web y backend              | T02, T03      |
| T05 | Campañas y sesiones de muestreo en backend                   | Backend                    | T02           |
| T06 | Aplicación móvil: navegación principal y perfil              | Aplicación móvil           | T01           |
| T07 | Aplicación móvil: taxonomía jerárquica, búsqueda y favoritos | Aplicación móvil y backend | T06, T01      |
| T08 | Aplicación móvil: creación de sesión de muestreo             | Aplicación móvil y backend | T05, T06      |
| T09 | API de observaciones y adjuntos para captura de campo        | Backend                    | T05, T07      |
| T10 | Aplicación móvil: formulario de observación                  | Aplicación móvil           | T07, T08, T09 |

La tabla anterior se incluye únicamente como síntesis de orientación. La planificación real del proyecto se entiende a partir de la relación entre las tareas, sus dependencias y el impacto que tienen en el producto final, como se describe en los apartados siguientes.

## **T01 – Login y logout común en app y portal**

La primera tarea tiene como objetivo funcional establecer un mecanismo común de inicio y cierre de sesión para los dos clientes del sistema: el portal web y la aplicación móvil. Su papel dentro del backlog es fundacional, ya que permite identificar al usuario antes de aplicar permisos, seleccionar grupos de trabajo o registrar datos científicos asociados a una persona concreta.

En el estado actual documentado del proyecto, esta tarea se materializa en los flujos de autenticación representados en los prototipos y en el diseño de la API REST, donde se definen los puntos de entrada de autenticación, renovación de sesión y cierre de sesión mediante tokens JWT. En el portal web se contempla la existencia de rutas públicas y protegidas, mientras que en la aplicación móvil el acceso al perfil y a las pantallas principales parte igualmente de una sesión activa. La dependencia de esta tarea es nula, por lo que actúa como punto de arranque del resto del Sprint 1.

Su impacto en el producto final es directo: proporciona la base de seguridad necesaria para que OmniLitter pueda diferenciar usuarios, asociar observaciones a investigadores concretos y mantener una experiencia coherente entre web y móvil. Además, permite que las tareas posteriores no dupliquen lógica de identificación en cada capa.

## **T02 – Roles, permisos y grupo activo**

La segunda tarea persigue introducir el modelo de autorización basado en roles y grupos. Su objetivo funcional es que el sistema no solo conozca qué usuario ha iniciado sesión, sino también qué permisos tiene dentro de cada grupo de investigación y cuál es el grupo activo sobre el que se filtran las operaciones.

La descripción incluida en la memoria se corresponde con el modelo de membresías definido en el diseño de datos, donde el rol no pertenece directamente al usuario, sino a su relación con un grupo determinado. Esta decisión permite que una misma persona pueda actuar como administradora en un grupo y como colaboradora en otro. En el portal web, el concepto de grupo activo condiciona la información mostrada en el dashboard y en los listados, mientras que las secciones de administración se reservan a perfiles con permisos suficientes.

La tarea depende de T01, porque la asignación de permisos solo puede aplicarse a usuarios autenticados. Su impacto es especialmente relevante para el producto final: garantiza el aislamiento de datos entre equipos de investigación, permite cumplir los requisitos de privacidad por grupo y prepara la base para la administración de usuarios, campañas y sesiones.

### **T03 – Portal web: layout, navegación y rutas protegidas**

Esta tarea tiene como objetivo funcional construir la estructura navegable del portal web. En términos de planificación, representa el paso desde la autenticación y el control de acceso hacia una interfaz organizada, capaz de separar las páginas públicas de las secciones internas del sistema.

El diseño documentado contempla un portal web de tipo *Single Page Application* con React, TypeScript, Vite y React Router. La navegación se organiza mediante rutas públicas, como la página de inicio y el formulario de acceso, y rutas protegidas bajo el área interna de la aplicación. Estas rutas protegidas verifican la existencia de una sesión activa y redirigen al usuario si no dispone de acceso. El layout general integra elementos persistentes, como barra lateral, cabecera y selector de grupo activo, que sirven de soporte para las pantallas posteriores.

T03 depende de T01 y T02, ya que la navegación protegida necesita autenticación y debe adaptarse a los permisos del usuario. Su impacto en el producto final consiste en proporcionar una experiencia web consistente, extensible y preparada para alojar el dashboard, los listados, la administración y los módulos de análisis que se incorporen en etapas posteriores.

### **T04 – Portal admin: usuarios, grupos y membresías**

La cuarta tarea amplía el portal web con capacidades administrativas. Su objetivo funcional es permitir la gestión de usuarios, grupos de investigación y membresías, de forma que la estructura organizativa del sistema pueda mantenerse desde el propio portal sin intervención directa sobre la base de datos.

El diseño documentado se apoya en el modelo de datos formado por usuarios, grupos y membresías, así como en el diseño de endpoints de la API para listar, crear, actualizar o archivar grupos y gestionar miembros. En la interfaz web, estas funciones se vinculan al rol de administrador y se ocultan a usuarios colaboradores, manteniendo la separación de responsabilidades dentro del sistema.

Esta tarea depende de T02, porque la administración solo tiene sentido una vez definido el modelo de roles, y de T03, porque necesita integrarse dentro de la navegación protegida del portal. Su impacto final es permitir que OmniLitter sea operable por equipos reales, facilitando la incorporación de investigadores, la creación de grupos y la asignación de permisos sin romper el aislamiento de datos.

### **T05 – Campañas y sesiones de muestreo en backend**

La quinta tarea introduce en el backend las entidades que estructuran la actividad científica del sistema: campañas y sesiones de muestreo. Su objetivo funcional es representar el

contexto en el que se capturan las observaciones, diferenciando el proyecto o campaña de investigación de cada salida concreta de campo.

En la documentación actual, esta tarea se refleja en el modelo relacional mediante las entidades `campaigns` y `sampling_sessions`, y en el contrato de API definido para crear, consultar y actualizar sesiones. Las sesiones almacenan metadatos relevantes como entorno, tipología, parámetros de muestreo, posición inicial, duración, versión de taxonomía y estado de sincronización. Esta información resulta necesaria para que los datos registrados sean interpretables y reproducibles.

T05 depende de T02, ya que las campañas y sesiones deben quedar asociadas a grupos y usuarios con permisos adecuados. Su impacto en el producto final es central: convierte la captura de residuos en un proceso científicamente contextualizado y prepara las dependencias necesarias para la creación de observaciones y adjuntos.

### **T06 – Aplicación móvil: navegación principal y perfil**

La sexta tarea traslada la base de autenticación a la aplicación móvil. Su objetivo funcional es establecer la navegación principal de la app y una pantalla de perfil que represente al usuario autenticado, sirviendo como punto de entrada para las funcionalidades de campo.

El diseño documentado plantea una aplicación móvil con React Native y Expo, organizada mediante navegación de primer nivel y flujos de pila. Las secciones principales se estructuran alrededor de la actividad de campo: inicio, sesiones, mapa, sincronización y perfil. Esta organización permite separar las acciones de captura de datos de las pantallas informativas y de configuración, manteniendo una interfaz adecuada para su uso en condiciones reales de muestreo.

T06 depende de T01 porque la navegación móvil parte de una sesión válida. Su impacto en el producto final consiste en proporcionar una estructura estable para que el usuario pueda iniciar sesiones, consultar su estado y acceder posteriormente a la taxonomía, la captura de observaciones y la sincronización.

### **T07 – Aplicación móvil: taxonomía jerárquica, búsqueda y favoritos**

La séptima tarea tiene como objetivo funcional incorporar en la aplicación móvil la taxonomía de residuos de forma navegable, con soporte para búsqueda y elementos favoritos. Esta funcionalidad resulta crítica porque la selección del tipo de residuo es uno de los datos principales de cada observación.

La memoria describe una taxonomía jerárquica basada en códigos estables y preparada para funcionar sin conexión. En el diseño de la aplicación móvil, dicha taxonomía se presenta mediante una navegación por categorías y elementos, complementada por mecanismos que

agilizan la captura en campo, como favoritos. En el backend, la taxonomía queda modelada mediante la entidad `taxonomy_items` y endpoints de consulta, de modo que pueda descargarse y asociarse a sesiones y observaciones con su versión correspondiente.

T07 depende de T06, porque se integra en la navegación móvil, y de T01, porque la descarga o disponibilidad de la taxonomía se vincula al contexto del usuario. Su impacto en el producto final es doble: mejora la usabilidad durante el trabajo de campo y refuerza la calidad científica de los datos al mantener una clasificación controlada y trazable.

### **T08 – Aplicación móvil: creación de sesión de muestreo**

La octava tarea permite iniciar desde la aplicación móvil una nueva sesión de muestreo. Su objetivo funcional es que el investigador pueda registrar el contexto de la salida de campo antes de introducir observaciones individuales.

El diseño documentado define un flujo de creación de sesión en el que se recogen datos como nombre, tipo de entorno, parámetros del transecto y coordenadas de inicio cuando estén disponibles. Esta información se relaciona con el diseño de persistencia local mediante SQLite, pensado para que las sesiones puedan crearse sin conexión y sincronizarse posteriormente con el backend. En la arquitectura general, la sesión actúa como contenedor de las observaciones generadas durante una jornada o tramo de muestreo.

T08 depende de T05, porque necesita el modelo backend de campañas y sesiones, y de T06, porque se integra dentro de la navegación móvil. Su impacto en el producto final es esencial: habilita el trabajo de campo de forma ordenada y permite asociar cada observación a un contexto metodológico claro.

### **T09 – API de observaciones y adjuntos para captura de campo**

La novena tarea se centra en la capa backend y tiene como objetivo funcional proporcionar la API necesaria para registrar observaciones y asociarles adjuntos, principalmente fotografías tomadas durante la captura de campo.

En el diseño actual, esta tarea se refleja en las entidades `observations`, `litter_items` y `observation_photos`, así como en los endpoints definidos para crear observaciones, listar observaciones de una sesión y subir fotografías mediante peticiones multipart. Las observaciones incluyen el tipo de residuo, cantidad, atributos descriptivos, coordenadas, fecha, comentarios, versión de taxonomía y estado de sincronización. Los adjuntos se modelan como recursos independientes para conservar metadatos y permitir una gestión posterior más flexible.

T09 depende de T05, porque toda observación debe pertenecer a una sesión, y de T07, porque el tipo de residuo procede de la taxonomía. Su impacto en el producto final es

convertir el backend en receptor de los datos científicos de campo, preparando la posterior visualización en el portal, la sincronización y las exportaciones.

### **T10 – Aplicación móvil: formulario de observación**

La décima tarea completa el primer flujo funcional de campo al incorporar en la aplicación móvil el formulario de observación. Su objetivo funcional es permitir que el usuario registre cada residuo observado con los datos mínimos necesarios para su análisis posterior.

La memoria recoge este flujo como una secuencia en la que el investigador selecciona un elemento de la taxonomía, introduce cantidad y atributos complementarios, añade comentarios y, cuando procede, incorpora una fotografía geolocalizada. El formulario se apoya en la navegación y en la sesión activa de la aplicación móvil, y se relaciona con el backend a través de la API de observaciones y adjuntos. En un escenario *offline-first*, los registros se almacenan localmente y se preparan para su sincronización diferida.

T10 depende de T07, T08 y T09, ya que necesita taxonomía, sesión activa y una API de persistencia para consolidar los datos. Su impacto en el producto final es especialmente visible: representa el punto en el que OmniLitter pasa de ser una estructura de gestión a una herramienta efectiva de captura de datos científicos en campo, dejando preparada la base para módulos posteriores como mapas, análisis, exportación y sincronización avanzada.

#### **5.3.0.2. Sprint 2: Captura avanzada de observaciones y flujo offline**

Durante el segundo sprint se abordaron las funcionalidades necesarias para completar el flujo de trabajo de campo en la aplicación móvil y permitir la posterior explotación de los datos desde el portal web. Este sprint se centró especialmente en la captura precisa de observaciones, el soporte offline, la gestión de fotografías y la validación de la información registrada.

**Cuadro 5.2:** Resumen del backlog del Sprint 2: captura avanzada y flujo offline.

| ID  | Funcionalidad                                                          | Área                       | Dependencias            |
|-----|------------------------------------------------------------------------|----------------------------|-------------------------|
| T11 | Aplicación móvil: GPS en sesión y observación                          | Aplicación móvil           | T08, T10                |
| T12 | Aplicación móvil: fotos por observación y meta-datos                   | Aplicación móvil y backend | T09, T10, T11           |
| T13 | Aplicación móvil: revisión, edición y eliminación antes de sincronizar | Aplicación móvil           | T08, T10, T12           |
| T14 | Backend: validación científica y permisos de observaciones             | Backend                    | T09, T10, T11, T12      |
| T15 | Aplicación móvil: persistencia offline del flujo de campo              | Aplicación móvil           | T07, T08, T10, T11, T12 |
| T16 | Portal web: listado de sesiones y observaciones con filtros            | Web y backend              | T04, T05, T09, T14      |
| T17 | Portal web: ficha de observación con fotos                             | Web                        | T12, T16                |

**T11 – GPS en sesión y observación**

|                     |                       |
|---------------------|-----------------------|
| <b>Área</b>         | Aplicación móvil      |
| <b>Responsable</b>  | Ignacio Martínez Díaz |
| <b>Prioridad</b>    | Crítica               |
| <b>Dependencias</b> | T08, T10              |

**Objetivo**

Integrar la captura de coordenadas GPS dentro del flujo de sesiones y observaciones, permitiendo registrar la localización exacta tanto del inicio de la sesión de muestreo como de cada observación individual.

**Valor aportado**

Permite georreferenciar la información científica recopilada y facilita su análisis posterior mediante herramientas cartográficas.

**T12 – Fotos por observación y metadatos**

|                     |                            |
|---------------------|----------------------------|
| <b>Área</b>         | Aplicación móvil + Backend |
| <b>Responsable</b>  | Ignacio Martínez Díaz      |
| <b>Prioridad</b>    | Alta                       |
| <b>Dependencias</b> | T09, T10, T11              |

**Objetivo**

Permitir la captura y asociación de fotografías a cada observación registrada, almacenando además metadatos relevantes como tamaño, formato, dimensiones, fecha de captura y otra información técnica asociada.

**Valor aportado**

Aporta evidencia visual a las observaciones y mejora los procesos de validación científica posteriores.

**T13 – Revisión, edición y eliminación antes de sincronizar**

|                     |                       |
|---------------------|-----------------------|
| <b>Área</b>         | Aplicación móvil      |
| <b>Responsable</b>  | Ignacio Martínez Díaz |
| <b>Prioridad</b>    | Alta                  |
| <b>Dependencias</b> | T08, T10, T12         |

**Objetivo**

Permitir al usuario revisar las observaciones almacenadas localmente antes de sincronizarlas con el servidor, incorporando capacidades de edición y eliminación.

**Valor aportado**

Mejora la calidad de los datos enviados al sistema central y reduce errores producidos durante la captura en campo.

**T14 – Validación científica y permisos de observaciones**

|                     |                    |
|---------------------|--------------------|
| <b>Área</b>         | Backend            |
| <b>Responsable</b>  | Alejandro          |
| <b>Prioridad</b>    | Crítica            |
| <b>Dependencias</b> | T09, T10, T11, T12 |

**Objetivo**

Implementar mecanismos de validación y autorización que garanticen la consistencia de los datos científicos y el cumplimiento de los permisos definidos para cada rol.

**Valor aportado**

Protege la integridad de la información registrada y evita modificaciones no autorizadas dentro del sistema.

**T15 – Persistencia offline del flujo de campo**

|                     |                         |
|---------------------|-------------------------|
| <b>Área</b>         | Aplicación móvil        |
| <b>Responsable</b>  | Ignacio Martínez Díaz   |
| <b>Prioridad</b>    | Crítica                 |
| <b>Dependencias</b> | T07, T08, T10, T11, T12 |

**Objetivo**

Garantizar el funcionamiento completo de la aplicación móvil en ausencia de conectividad mediante almacenamiento local de sesiones, observaciones, fotografías y coordenadas GPS.

**Valor aportado**

Permite utilizar OmniLitter en entornos remotos donde no existe cobertura de red estable, manteniendo la continuidad del trabajo de campo.

**T16 – Listado de sesiones y observaciones con filtros**

|                     |                    |
|---------------------|--------------------|
| <b>Área</b>         | Web + Backend      |
| <b>Responsable</b>  | Alejandro          |
| <b>Prioridad</b>    | Alta               |
| <b>Dependencias</b> | T04, T05, T09, T14 |

**Objetivo**

Desarrollar una interfaz web que permita consultar sesiones y observaciones registradas mediante filtros y mecanismos de búsqueda avanzados.

**Valor aportado**

Facilita el acceso a la información recopilada y mejora las capacidades de análisis por parte de investigadores y administradores.

**T17 – Ficha de observación con fotografías**

|                     |           |
|---------------------|-----------|
| <b>Área</b>         | Web       |
| <b>Responsable</b>  | Alejandro |
| <b>Prioridad</b>    | Alta      |
| <b>Dependencias</b> | T12, T16  |

**Objetivo**

Implementar una vista detallada de observación en el portal web que permita consultar todos los atributos registrados junto con las fotografías asociadas.

**Valor aportado**

Facilita la validación visual de los registros y completa el ciclo de captura, sincronización y revisión de observaciones.

## 5.4. Gestión de reuniones

---

Las reuniones con el cliente han tenido un papel central en la definición del alcance. La primera reunión permitió identificar el problema y las funcionalidades esperadas. Posteriormente, una segunda reunión sirvió para revisar los prototipos, confirmar la cobertura de los flujos principales y reforzar prioridades como el funcionamiento sin conexión, la separación por grupos y la trazabilidad científica.

Estas reuniones se documentan en la Sección 4 y en el Anexo B, donde se relacionan las necesidades detectadas con los requisitos del sistema.

## 5.5. Gestión de riesgos

---

## 5.6. Gestión del alcance

---

El alcance se ha dividido en dos niveles:

- **Alcance funcional validado mediante prototipos:** portal web y aplicación móvil con flujos navegables, datos de ejemplo, roles simulados y pantallas principales implementadas.
- **Alcance de implementación final:** backend REST, base de datos, autenticación, sincronización offline, persistencia de observaciones y conexión progresiva de los clientes al backend.

Esta separación permite demostrar primero la experiencia de uso y después construir la infraestructura técnica necesaria para convertir el prototipo en una aplicación real.

## 6. Tecnologías utilizadas

En este capítulo se describen las tecnologías seleccionadas para el desarrollo de OmniLitter y la justificación de su elección. La selección se ha realizado teniendo en cuenta tres restricciones principales: el alcance de un Trabajo de Fin de Grado desarrollado por dos personas, la necesidad de disponer de una aplicación móvil capaz de trabajar sin conexión y la conveniencia de mantener un stack homogéneo basado en TypeScript.

### 6.1. Criterios de selección

---

Los criterios utilizados para seleccionar las tecnologías han sido los siguientes:

- **Homogeneidad tecnológica:** se prioriza el uso de TypeScript tanto en el frontend como en el backend para reducir el cambio de contexto durante el desarrollo.
- **Capacidad offline:** la aplicación móvil debe poder crear sesiones, observaciones y fotografías sin depender de la conexión a Internet.
- **Madurez del ecosistema:** se eligen tecnologías ampliamente usadas, con documentación suficiente y comunidad activa.
- **Facilidad de despliegue:** el sistema debe poder desplegarse en servicios accesibles para un proyecto académico, como Vercel, Render, Railway o una máquina virtual sencilla.
- **Escalabilidad razonable:** aunque el proyecto no persigue una escala industrial, el diseño debe permitir incorporar nuevos grupos, campañas y datos de campo sin rehacer la arquitectura.

**Tecnologías empleadas en los prototipos** Durante la fase inicial se construyeron prototipos funcionales con datos en memoria: un portal web y una simulación de la aplicación móvil en navegador. Estos prototipos permitieron validar flujos de usuario con el cliente antes de trasladar las decisiones principales a la implementación final.

**Cuadro 6.1:** Tecnologías empleadas en los prototipos funcionales.

| Tecnología                    | Uso en el proyecto                                                             |
|-------------------------------|--------------------------------------------------------------------------------|
| React 18                      | Construcción de interfaces de usuario para el portal web y el prototipo móvil. |
| TypeScript                    | Tipado estático de componentes, rutas, estado y datos de ejemplo.              |
| Vite                          | Servidor de desarrollo y empaquetado rápido de los prototipos.                 |
| Tailwind CSS                  | Sistema de estilos utilitario para construir pantallas responsive.             |
| React Router                  | Definición de rutas públicas, rutas protegidas y navegación entre pantallas.   |
| Recharts                      | Gráficas y visualizaciones estadísticas del dashboard del portal web.          |
| Lucide React / Material Icons | Iconografía de la interfaz.                                                    |

## 6.2. Portal web

El portal web se plantea como una aplicación *Single Page Application* desarrollada con React, TypeScript y Vite. Su responsabilidad principal es permitir a investigadores y administradores consultar observaciones, gestionar usuarios y grupos, visualizar mapas y exportar datos.

**Cuadro 6.2:** Stack seleccionado para el portal web.

| Tecnología              | Responsabilidad                                                           |
|-------------------------|---------------------------------------------------------------------------|
| React                   | Construcción de componentes reutilizables y pantallas del portal.         |
| TypeScript              | Definición de tipos de dominio, respuestas de API y props de componentes. |
| Vite                    | Compilación, servidor de desarrollo y generación del bundle final.        |
| Tailwind CSS            | Sistema visual responsive y consistente con los prototipos.               |
| React Router            | Gestión de rutas públicas y protegidas.                                   |
| Axios / Fetch API       | Comunicación con la API REST del backend.                                 |
| Leaflet / React Leaflet | Visualización geográfica de observaciones en mapas interactivos.          |
| Recharts                | Dashboards, rankings y tendencias temporales.                             |

### 6.3. Aplicación móvil

La aplicación móvil se plantea con React Native y Expo, utilizando SQLite mediante el paquete `expo-sqlite` como base de datos local. El objetivo es que el usuario pueda completar una jornada de muestreo aunque el dispositivo no tenga conexión a Internet.

**Cuadro 6.3:** Stack seleccionado para la aplicación móvil.

| Tecnología                                                         | Responsabilidad                                                                                 |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| React Native                                                       | Desarrollo multiplataforma para Android e iOS.                                                  |
| Expo                                                               | Toolchain de desarrollo, compilación y acceso a módulos nativos.                                |
| TypeScript                                                         | Tipado de pantallas, servicios, modelos locales y respuestas de API.                            |
| <code>expo-sqlite</code> / SQLite                                  | Persistencia local de sesiones, observaciones, fotografías pendientes y cola de sincronización. |
| <code>expo-location</code>                                         | Captura de coordenadas GPS asociadas a sesiones y observaciones.                                |
| <code>expo-camera</code>                                           | Captura de fotografías y obtención de metadatos cuando sea posible.                             |
| <code>expo-task-manager</code> / <code>expo-background-task</code> | Ejecución de tareas de sincronización en segundo plano cuando el sistema operativo lo permita.  |
| React Navigation                                                   | Navegación entre pantallas principales y flujos de captura.                                     |

### 6.4. Backend y persistencia

El backend centraliza la lógica de negocio, la autenticación, el control de acceso por grupo y la sincronización de datos procedentes de dispositivos móviles. Se plantea como una API REST desarrollada con Node.js, Express y TypeScript.

**Cuadro 6.4:** Stack seleccionado para backend y persistencia.

| Tecnología                          | Responsabilidad                                                      |
|-------------------------------------|----------------------------------------------------------------------|
| Node.js                             | Entorno de ejecución del backend.                                    |
| Express                             | Definición de rutas, middlewares y controladores REST.               |
| TypeScript                          | Tipado de servicios, DTOs, entidades y errores.                      |
| PostgreSQL                          | Base de datos relacional principal del sistema.                      |
| Prisma                              | ORM para modelar entidades, migraciones y acceso a datos.            |
| JWT                                 | Autenticación mediante tokens de acceso y refresco.                  |
| bcrypt                              | Almacenamiento seguro de contraseñas mediante hash.                  |
| multer / almacenamiento de ficheros | Recepción y almacenamiento de fotografías asociadas a observaciones. |

## 6.5. Despliegue y herramientas auxiliares

## 6.6. Alternativas consideradas

**Justificación de SQLite frente a WatermelonDB** Aunque inicialmente se valoró WatermelonDB como solución local *offline-first*, se ha decidido emplear SQLite directamente para ajustar mejor el alcance del TFG. SQLite permite cubrir los requisitos principales de la aplicación móvil: almacenar sesiones, observaciones, fotografías pendientes y una cola de sincronización local.

La decisión reduce la complejidad técnica, evita depender de una capa adicional de sincronización y facilita que los autores controlen de forma explícita el esquema local y el flujo de subida de datos. El patrón *offline-first* se mantiene, pero se implementa mediante una tabla local de cola de sincronización y servicios propios de lectura/escritura sobre SQLite.

## 7. Diseño de la solución

Este capítulo describe las decisiones de diseño adoptadas para materializar los requisitos funcionales y no funcionales de OmniLitter en una arquitectura concreta. Se abordan, en orden de abstracción descendente, la arquitectura general del sistema, el diseño de la base de datos, el contrato de la API REST, la estructura de componentes del portal web y de la aplicación móvil, y el mecanismo de sincronización *offline*.

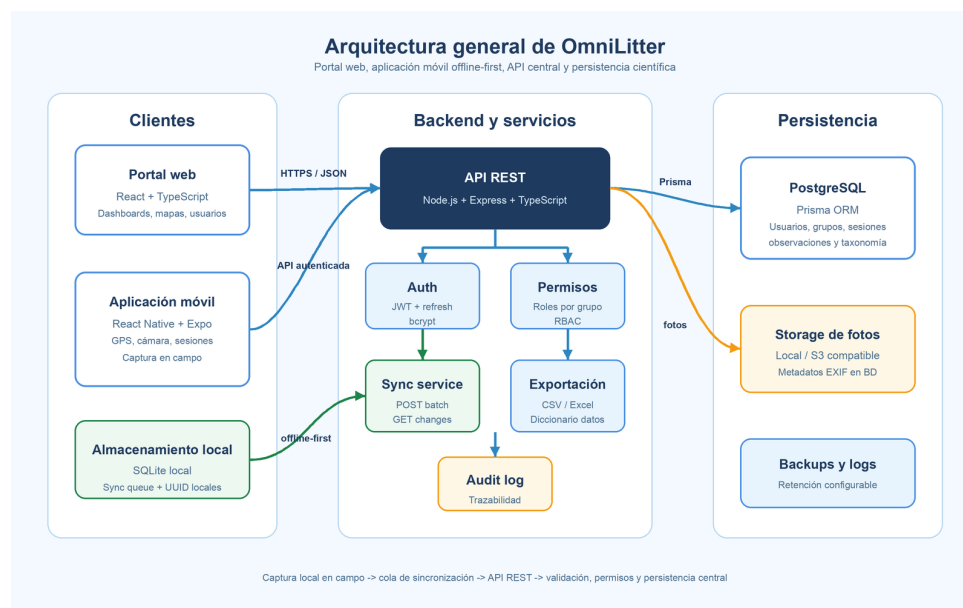
### 7.1. Arquitectura general

---

OmniLitter sigue una arquitectura de **tres capas** clásica adaptada a un contexto multi-plataforma con requisito de operación sin conexión:

1. **Capa de presentación.** Comprende dos clientes independientes: el portal web, orientado a la consulta y gestión de datos por parte de investigadores y administradores, y la aplicación móvil, orientada a la captura de observaciones en campo.
2. **Capa de lógica de negocio.** Una API REST centralizada planteada con Node.js + Express actúa como único punto de acceso a los datos, aplica las reglas de negocio y gestiona la autenticación mediante *tokens* JWT.
3. **Capa de persistencia.** PostgreSQL almacena el estado canónico del sistema. Adicionalmente, la aplicación móvil mantiene una base de datos local SQLite que permite operar sin conexión y sincronizarse en diferido.

La figura 7.1 muestra un diagrama de la arquitectura del sistema.



**Figura 7.1:** Arquitectura general propuesta para OmniLitter.

### 7.1.1. Justificación de las decisiones arquitectónicas principales

**API REST como único punto de acceso.** Centralizar la lógica de negocio en la API evita duplicidades entre los dos clientes y facilita la incorporación de nuevos consumidores en el futuro (p.ej., exportaciones automatizadas por *scripts* externos o integraciones con repositorios científicos).

**Separación entre portal web y aplicación móvil.** Aunque comparten la misma API, los dos clientes tienen requisitos de experiencia de usuario muy distintos: el portal prioriza la visualización analítica y la gestión, mientras que la aplicación móvil prioriza la captura rápida en condiciones de campo adversas (luz solar, guantes, sin conexión). Mantenerlos como proyectos independientes permite evolucionar cada uno a su propio ritmo.

**Base de datos local en el móvil.** El trabajo de campo se realiza habitualmente en zonas costeras, ribereñas o forestales con cobertura de red intermitente o nula. La decisión de embeber una base de datos local en el dispositivo es, por tanto, un requisito no negociable para la viabilidad del producto.

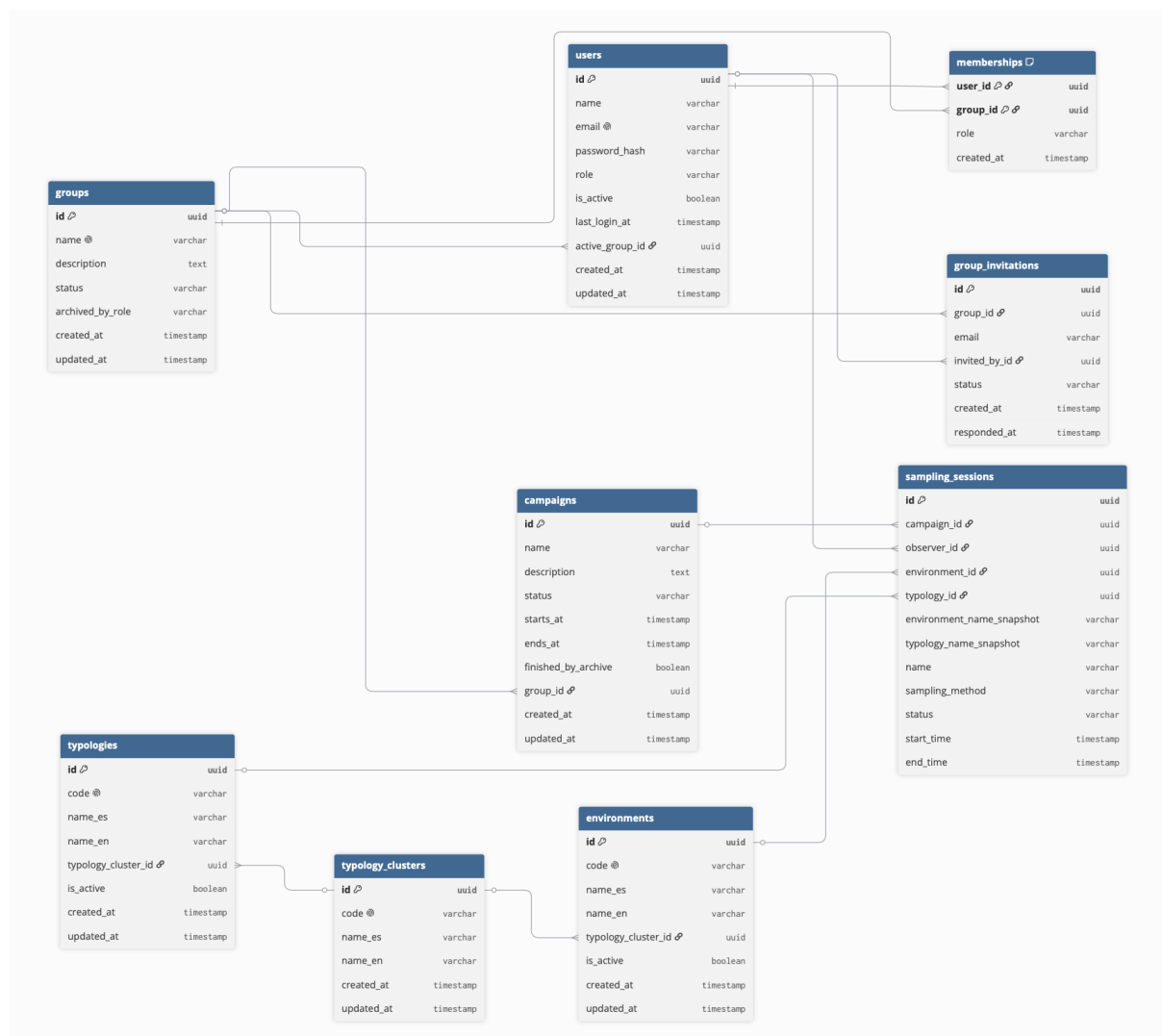
## 7.2. Diseño de la base de datos

### 7.2.1. Modelo relacional

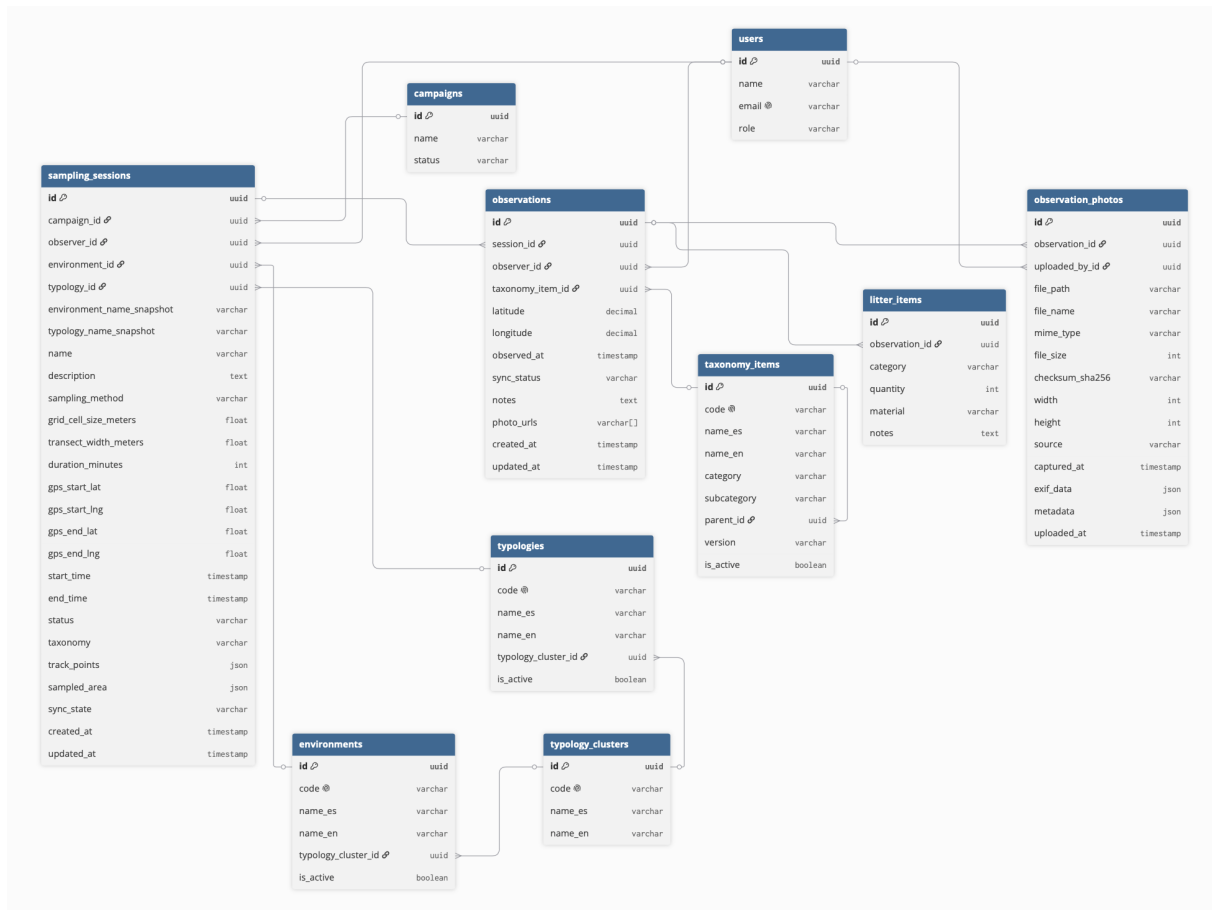
El modelo de datos final de OmniLitter está formado por quince entidades. Para que el diagrama resulte legible y sean más visibles las relaciones, se ha dividido en tres bloques funcionales, cada uno con su propio diagrama entidad-relación:

- **Usuarios, grupos, campañas y catálogo:** `users`, `groups`, `memberships`, `group_invitations`, `campaigns`, `environments`, `typologies` y `typology_clusters`. Recoge la gestión de acceso, la pertenencia a grupos de investigación, las invitaciones, la organización de campañas y el catálogo global de entornos y tipologías.
- **Muestreo, observaciones y taxonomía:** `sampling_sessions`, `observations`, `litter_items`, `observation_photos` y `taxonomy_items`. Es el núcleo operativo del sistema, donde se registran las sesiones de campo, las observaciones de residuos, su desglose y clasificación y las fotografías asociadas.
- **Autenticación y seguridad:** `users`, `refresh_tokens` y `password_reset_tokens`. Agrupa las entidades que dan soporte al inicio de sesión, la renovación de tokens y el restablecimiento de contraseña.

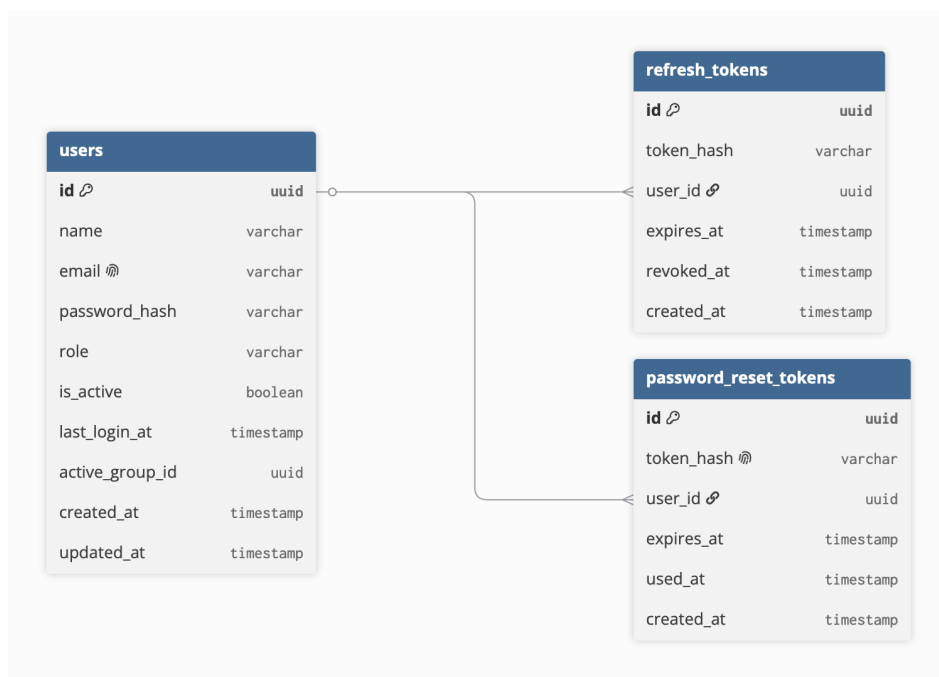
La entidad `sampling_sessions` aparece en los dos primeros bloques porque sirve de unión entre la organización de campañas y la captura de datos en campo. Las figuras 7.2, 7.3 y 7.4 muestran cada uno de estos bloques.



**Figura 7.2:** Modelo entidad-relación. Bloque de usuarios, grupos, campañas y catálogo de entornos.



**Figura 7.3:** Modelo entidad-relación. Bloque de sesiones de muestreo, observaciones, residuos, fotografías y taxonomía.



**Figura 7.4:** Modelo entidad-relación. Bloque de autenticación y seguridad.

### 7.2.2. Descripción de entidades

La tabla 7.1 describe los atributos clave de cada entidad y su propósito en el sistema.

**Cuadro 7.1:** Descripción de entidades del modelo de datos.

| Entidad           | Atributos principales                                                                 | Descripción                                                                                                                                                                                                                                                         |
|-------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| users             | id, name, email, password_hash, role, is_active, last_login_at, active_group_id (FK)  | Persona registrada en el sistema. La contraseña se guarda como <i>hash</i> y nunca en texto plano. El campo <b>role</b> indica el rol global (SUPERADMIN o USUARIO) y <b>active_group_id</b> apunta al grupo con el que el usuario está trabajando en cada momento. |
| groups            | id, name, description, status, archived_by_role                                       | Grupo de investigación. Un grupo puede archivarse sin borrar sus datos históricos, lo que conserva la trazabilidad de las campañas. El campo <b>archived_by_role</b> registra el rol que lo archivó, para condicionar quién puede restaurarlo.                      |
| memberships       | user_id (FK), group_id (FK), role, created_at                                         | Tabla de unión entre usuarios y grupos, con clave primaria compuesta. El campo <b>role</b> toma los valores ADMIN o FIELD_RESEARCHER, de modo que un mismo usuario puede tener un rol distinto en cada grupo.                                                       |
| group_invitations | id, group_id (FK), email, invited_by_id (FK), status, responded_at                    | Invitación de un usuario a un grupo a partir de su correo. El campo <b>status</b> sigue el ciclo PENDING, ACCEPTED o REJECTED.                                                                                                                                      |
| campaigns         | id, name, description, status, starts_at, ends_at, finished_by_archive, group_id (FK) | Proyecto de muestreo asociado a un grupo. Agrupa varias sesiones de campo bajo un objetivo común. El estado evoluciona entre DRAFT, ACTIVE, FINISHED y ARCHIVED.                                                                                                    |
| typology_cluster  | id, code, name_es, name_en                                                            | Agrupación que relaciona un entorno con el conjunto de tipologías compatibles. Al existir como entidad propia, los entornos y las tipologías se asocian mediante clave foránea en lugar de por texto.                                                               |

| Entidad           | Atributos principales                                                                                                                                                                                                                                                                                                                       | Descripción                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| environments      | id, code, name_es, name_en, typology_cluster_id (FK), is_active                                                                                                                                                                                                                                                                             | Catálogo global de tipos de entorno de muestreo, como playa, ribera, bosque, carretera o campo de cultivo. Cada entorno cuelga de un <code>typology_cluster</code> y dispone de nombre en español e inglés.                                                                                                                                                                                                                                                                |
| typologies        | id, code, name_es, name_en, typology_cluster_id (FK), is_active                                                                                                                                                                                                                                                                             | Catálogo global de subentornos o tipologías. Comparten <code>typology_cluster</code> con su entorno, lo que permite ofrecer en la app solo las tipologías coherentes con el entorno elegido.                                                                                                                                                                                                                                                                               |
| sampling_sessions | id, campaign_id (FK), observer_id (FK), environment_id (FK), typology_id (FK), environment_name_snapshot, typology_name_snapshot, name, sampling_method, grid_cell_size_meters, transect_width_meters, duration_minutes, gps_start_lat/lng, gps_end_lat/lng, start_time, end_time, status, taxonomy, track_points, sampled_area, sync_state | Sesión de muestreo realizada en campo. Referencia el entorno y la tipología del catálogo y, al mismo tiempo, guarda su nombre en los campos <i>snapshot</i> . Registra el método de muestreo (LIBRE, CUADRICULA o TRANSECTO), sus parámetros, la duración y las coordenadas de inicio y fin. Los campos <code>track_points</code> y <code>sampled_area</code> almacenan el recorrido y el área muestreada, y <code>sync_state</code> controla el estado de sincronización. |
| observations      | id, session_id (FK), observer_id (FK), taxonomy_item_id (FK), latitude, longitude, observed_at, sync_status, notes, photo_urls                                                                                                                                                                                                              | Registro de un residuo avistado dentro de una sesión. Conserva la posición ( <code>latitude</code> y <code>longitude</code> ), el instante de captura y la referencia a la taxonomía. Es la unidad básica de dato científico del sistema.                                                                                                                                                                                                                                  |

| Entidad                            | Atributos principales                                                                                                                                                                                                                                                                                                                                               | Descripción                                                                                                                                                                                                                              |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>litter_items</code>          | <code>id</code> , <code>observation_id</code> (FK), <code>category</code> , <code>quantity</code> , <code>material</code> , <code>notes</code>                                                                                                                                                                                                                      | Desglose de los residuos contabilizados en una observación. El campo <code>category</code> usa la clasificación por material (PLASTIC, METAL, GLASS y demás categorías) y <code>quantity</code> recoge la cantidad como entero positivo. |
| <code>observation_photos</code>    | <code>id</code> , <code>observation_id</code> (FK), <code>uploaded_by_id</code> (FK), <code>file_path</code> , <code>file_name</code> , <code>mime_type</code> , <code>file_size</code> , <code>checksum_sha256</code> , <code>width</code> , <code>height</code> , <code>source</code> , <code>captured_at</code> , <code>exif_data</code> , <code>metadata</code> | Fotografía asociada a una observación. Almacena los datos del fichero, un <code>checksum</code> SHA-256 para verificar la integridad tras la sincronización y los metadatos EXIF de la imagen cuando están disponibles.                  |
| <code>taxonomy_items</code>        | <code>id</code> , <code>code</code> , <code>name_es</code> , <code>name_en</code> , <code>category</code> , <code>subcategory</code> , <code>parent_id</code> (FK), <code>version</code> , <code>is_active</code>                                                                                                                                                   | Catálogo jerárquico de tipos de residuo basado en listas comunes de categorías. La referencia <code>parent_id</code> hacia la propia entidad permite organizar la taxonomía por niveles y descargarla completa para usarla sin conexión. |
| <code>refresh_tokens</code>        | <code>id</code> , <code>token_hash</code> , <code>user_id</code> (FK), <code>expires_at</code> , <code>revoked_at</code>                                                                                                                                                                                                                                            | Token de renovación de sesión. Se guarda únicamente su <i>hash</i> , junto con la fecha de expiración y, en su caso, la de revocación.                                                                                                   |
| <code>password_reset_tokens</code> | <code>id</code> , <code>token_hash</code> , <code>user_id</code> (FK), <code>expires_at</code> , <code>used_at</code>                                                                                                                                                                                                                                               | Token de restablecimiento de contraseña. Es de un solo uso: el campo <code>used_at</code> marca el momento en que se consume y lo invalida.                                                                                              |

### 7.2.3. Decisiones de diseño de la base de datos

**UUID como clave primaria en registros sincronizables.** Los registros que pueden crearse en el dispositivo móvil antes de comunicarse con el servidor utilizan identificadores UUID generados en el propio cliente. Esto resuelve las colisiones de claves en escenarios de creación offline: dos dispositivos que creen una sesión o una observación sin conexión obtienen identificadores únicos que no chocan al sincronizarse con el servidor.

**Modelo de roles en dos niveles.** El control de acceso separa el rol global del usuario,

almacenado en `users.role` con los valores `SUPERADMIN` o `USUARIO`, del rol que tiene dentro de cada grupo, almacenado en `memberships.role` como `ADMIN` o `FIELD RESEARCHER`. El rol operativo no es una propiedad fija de la persona, sino de su relación con un grupo concreto, por lo que un mismo investigador puede administrar un grupo y participar como investigador de campo en otro. El superadmin queda reservado a la administración global de la plataforma.

**Catálogo normalizado de entornos con copia histórica en la sesión.** El catálogo de entornos y tipologías se modela con tres entidades relacionadas por clave foránea, no por texto. La entidad `typology_clusters` agrupa cada entorno con las tipologías compatibles, y tanto `environments` como `typologies` cuelgan del mismo *cluster*. Así, al elegir un entorno la aplicación ofrece solo las tipologías coherentes con él. Cada registro del catálogo guarda su nombre en español e inglés (`name_es` y `name_en`), lo que permite al superadmin configurar ambos idiomas desde el portal.

Cuando se crea una sesión, esta referencia el entorno y la tipología mediante clave foránea real (`environment_id` y `typology_id`) y, al mismo tiempo, copia su nombre en los campos `environment_name_snapshot` y `typology_name_snapshot`. La clave foránea mantiene la integridad con el catálogo vigente, mientras que la copia conserva el valor utilizado en el momento del muestreo aunque el catálogo cambie después. Esta combinación evita inconsistencias en los datos históricos y simplifica la sincronización de la app, que no necesita resolver referencias contra un catálogo que pudo cambiar mientras el dispositivo estaba sin conexión.

**Separación entre observación y desglose de residuos.** Una observación representa un punto avistado, con sus coordenadas e instante de captura, mientras que `litter_items` recoge los residuos contabilizados en ese punto, con su categoría por material y su cantidad. Separar ambos conceptos permite asociar varios tipos de residuo a una misma observación sin duplicar la información de posición, y deja la clasificación detallada en manos de `taxonomy_items`.

**Precisión geográfica con tipo decimal.** Las coordenadas de las observaciones se almacenan con tipo `decimal` de precisión fija (nueve dígitos, seis de ellos decimales) en lugar de coma flotante. Así se obtiene una precisión cercana a los diez centímetros y se evitan los errores de redondeo propios del punto flotante, algo necesario para una herramienta cuya utilidad depende de la calidad de la georreferenciación.

**Estados de sincronización en sesiones y observaciones.** Tanto `sampling_sessions` como `observations` incluyen un campo de estado de sincronización basado en el enum `SyncStatus` (`SYNCED`, `PENDING`, `FAILED` o `CONFLICT`). Este campo sostiene el flujo de trabajo sin conexión, ya que permite distinguir qué registros están pendientes de subir, cuáles se han confirmado en el servidor y cuáles han fallado.

**Estrategias de borrado según el tipo de relación.** El esquema aplica reglas de borrado coherentes con el valor de cada relación. Los datos estrictamente dependientes se

eliminan en cascada junto a su contenedor: las membresías e invitaciones de un grupo, las sesiones de una campaña y las observaciones, residuos y fotos asociados a una sesión. En cambio, las referencias al autor (`observer_id` y `uploaded_by_id`) usan una regla restrictiva, de modo que no se puede borrar un usuario que tenga datos de campo asociados y se preserva la autoría. Las campañas pueden quedar sin grupo mediante `SetNull` para mantener el histórico si se elimina el grupo, y el ítem taxonómico de una observación también se puede poner a nulo para no perder el registro científico.

En el catálogo ambiental se combinan restricciones y copia histórica. El entorno de una sesión usa una relación restrictiva, porque toda sesión debe conservar un contexto ambiental válido. La tipología puede ponerse a nulo si se elimina del catálogo, pero la sesión mantiene `typology_name_snapshot`; de forma equivalente, `environment_name_snapshot` conserva el nombre visible del entorno utilizado en campo. Así se evita que las tareas de mantenimiento del catálogo degraden los datos ya registrados.

**JSON para datos de estructura variable.** Los campos `track_points`, `sampled_area`, `exif_data` y `metadata` se almacenan como JSON. El recorrido y el área muestreada tienen una geometría que depende del método de muestreo, y los metadatos EXIF varían según el modelo de cámara. Guardar estos datos como JSON evita multiplicar columnas opcionales y permite ampliar la información asociada sin migrar el esquema.

### 7.3. Diseño de la API REST

---

La API sigue los principios REST: interfaz uniforme, comunicación sin estado, recursos identificados por URI y representaciones en formato JSON. Todos los *endpoints* protegidos requieren el encabezado `Authorization: Bearer <token>`, donde el *token* es un JWT firmado emitido en el *endpoint* de autenticación.

El cuadro 7.2 recoge los *endpoints* principales agrupados por módulo funcional. En lugar de presentar todas las rutas en una única tabla continua, se separan por bloque para que resulte más visible qué parte de la API atiende cada conjunto de operaciones.

**Cuadro 7.2:** Principales *endpoints* de la API REST de OmniLitter.

## Autenticación

| HTTP | Ruta                       | Acceso  | Uso                                          |
|------|----------------------------|---------|----------------------------------------------|
| POST | /api/auth/login            | Público | Emite JWT de acceso y refresco               |
| POST | /api/auth/register         | Público | Registra una cuenta nueva                    |
| POST | /api/auth/recover-password | Público | Inicia la recuperación de contraseña         |
| POST | /api/auth/reset-password   | Público | Restablece la contraseña con un <i>token</i> |
| POST | /api/auth/refresh          | Público | Renueva el JWT de acceso                     |
| POST | /api/auth/logout           | Usuario | Invalida el <i>token</i> de refresco         |

## Usuarios

| HTTP  | Ruta                 | Acceso  | Uso                               |
|-------|----------------------|---------|-----------------------------------|
| GET   | /api/users           | Admin   | Lista todos los usuarios          |
| POST  | /api/users           | Admin   | Crea un usuario                   |
| GET   | /api/users/me        | Usuario | Datos del usuario autenticado     |
| GET   | /api/users/me/groups | Usuario | Grupos del usuario y grupo activo |
| PATCH | /api/users/:id       | Admin   | Actualiza datos, rol o estado     |

## Grupos e invitaciones

| HTTP | Ruta                              | Acceso  | Uso                              |
|------|-----------------------------------|---------|----------------------------------|
| GET  | /api/groups                       | Usuario | Lista grupos del usuario         |
| POST | /api/groups                       | Admin   | Crea un grupo                    |
| PUT  | /api/groups/:id                   | Admin   | Edita un grupo                   |
| GET  | /api/groups/:id/backup            | Admin   | Descarga copia de seguridad JSON |
| POST | /api/group-invitations/:id/accept | Usuario | Acepta una invitación            |

## Campañas

| HTTP | Ruta               | Acceso  | Uso                             |
|------|--------------------|---------|---------------------------------|
| GET  | /api/campaigns     | Usuario | Lista campañas del grupo activo |
| POST | /api/campaigns     | Admin   | Crea una campaña                |
| PUT  | /api/campaigns/:id | Admin   | Edita una campaña               |

## Catálogo científico

| HTTP | Ruta                      | Acceso     | Uso                           |
|------|---------------------------|------------|-------------------------------|
| GET  | /api/catalog/environments | Usuario    | Lista entornos del catálogo   |
| POST | /api/catalog/environments | Superadmin | Crea un entorno               |
| GET  | /api/catalog/typologies   | Usuario    | Lista tipologías del catálogo |

## Sesiones de muestreo

| HTTP  | Ruta                 | Acceso  | Uso                                  |
|-------|----------------------|---------|--------------------------------------|
| GET   | /api/sessions        | Usuario | Lista sesiones accesibles            |
| POST  | /api/sessions        | Usuario | Crea una sesión                      |
| GET   | /api/sessions/:id    | Usuario | Obtiene una sesión con observaciones |
| GET   | /api/sessions/tracks | Usuario | Recorridos y áreas muestreadas       |
| PATCH | /api/sessions/:id    | Usuario | Actualiza metadatos o estado         |

## Observaciones y fotos

| HTTP | Ruta                                   | Acceso  | Uso                             |
|------|----------------------------------------|---------|---------------------------------|
| GET  | /api/observations                      | Usuario | Lista observaciones con filtros |
| POST | /api/observations                      | Usuario | Registra una observación        |
| GET  | /api/observations/:id/photos           | Usuario | Lista fotos de una observación  |
| POST | /api/observations/:id/photos           | Usuario | Sube una foto <i>multipart</i>  |
| GET  | /api/observations/:id/photos/:pid/file | Usuario | Sirve el archivo de imagen      |

## Taxonomía

| HTTP | Ruta                | Acceso  | Uso                                         |
|------|---------------------|---------|---------------------------------------------|
| GET  | /api/taxonomy/items | Usuario | Descarga la taxonomía filtrable por versión |

## Métricas

| HTTP | Ruta                      | Acceso  | Uso                                  |
|------|---------------------------|---------|--------------------------------------|
| GET  | /api/metrics              | Usuario | Métricas agregadas para el dashboard |
| GET  | /api/metrics/observations | Usuario | Datos por observación para analítica |

## Sincronización

| HTTP | Ruta              | Acceso  | Uso                                             |
|------|-------------------|---------|-------------------------------------------------|
| POST | /api/sync/batch   | Usuario | Sube el lote de registros pendientes            |
| GET  | /api/sync/changes | Usuario | Descarga cambios desde la última sincronización |

La exportación científica no dispone de un *endpoint* propio: el portal web genera los ficheros CSV (observaciones, ítems, resumen y un formato científico con diccionario de columnas) y GeoJSON a partir de los datos ya consultados. El contrato completo de la API se documenta con OpenAPI y puede explorarse de forma interactiva en `/api/docs` mediante Swagger UI.

## 7.4. Seguridad y control de acceso

---

El sistema se diseña con autenticación basada en JWT y dos *tokens*: un **token de acceso** de corta duración (15 minutos) y un **token de refresco** de larga duración (7 días). El token de acceso se envía en cada petición en el encabezado **Authorization**; el de refresco se utiliza exclusivamente para solicitar un nuevo token de acceso cuando el anterior expira.

La autorización sigue un modelo de control de acceso basado en roles (RBAC) a nivel de grupo. El *middleware* de autorización verifica, para cada petición, que el usuario autenticado posee el rol requerido en el grupo al que pertenece el recurso solicitado. Esto permite que una misma persona sea administradora en un grupo y colaboradora en otro, con permisos diferentes en cada contexto.

## 7.5. Diseño del portal web

---

### 7.5.1. Arquitectura de componentes

El portal web se diseña como una *Single Page Application* (SPA) con React 18 y TypeScript, empaquetada con Vite y estilizada con Tailwind CSS. El enrutado se resuelve en el cliente con la API History del navegador (`pushState` y `replaceState`), sin una librería de enrutado externa, y distingue dos niveles de rutas:

- **Rutas públicas:** la página de inicio (/) y el formulario de inicio de sesión (/login), accesibles sin autenticación.
- **Rutas protegidas:** las páginas privadas bajo /app/\*, que comprueban el estado de autenticación antes de renderizarse y redirigen al *login* si no existe una sesión activa.

La estructura de componentes sigue una jerarquía de tres niveles: el **Layout** general (barra lateral, encabezado, selector de grupo activo) envuelve las páginas individuales, que a su vez utilizan componentes reutilizables (tarjetas de estadística, tablas filtrables, modales de confirmación). Las vistas geográficas se construyen con Leaflet a través de **react-leaflet**, con agrupación de marcadores y una capa de mapa de calor, y los gráficos del dashboard y de la analítica se generan con Recharts.

### 7.5.2. Gestión de estado

El portal no utiliza una librería de gestión de estado externa. El estado de la aplicación se mantiene en componentes React y se transmite mediante propiedades, algo suficiente para la escala del portal. La sesión del usuario se conserva entre recargas en el almacenamiento del navegador, donde se guardan los *tokens* de acceso y refresco emitidos por el backend, y se renueva de forma automática cuando el token de acceso expira.

El grupo activo forma parte de ese estado de sesión. Al cambiarlo, las vistas que dependen del grupo (campanas, observaciones, mapa, dashboard y analítica) recargan su contenido para reflejar únicamente los datos del grupo seleccionado.

### 7.5.3. Flujo de navegación y control de acceso por rol

Las secciones de gestión de grupos y usuarios están reservadas al rol de administrador. El control efectivo de acceso lo aplica el backend, que rechaza cualquier operación no autorizada aunque la vista llegue a cargarse; la interfaz, por su parte, oculta o deshabilita las acciones de administración para los colaboradores. El Dashboard ajusta el contenido según el rol: los administradores ven las estadísticas del grupo activo, mientras que los colaboradores ven sus propias métricas. La tabla de observaciones muestra a cada colaborador sus registros y al administrador los de todo el grupo.

## 7.6. Diseño de la aplicación móvil

---

### 7.6.1. Stack tecnológico y estructura de pantallas

La aplicación móvil está planteada con **React Native** y **Expo**, usando *development builds* o *prebuild* cuando sea necesario acceder a capacidades nativas del dispositivo. Esta aproximación permite mantener las ventajas del ecosistema Expo sin renunciar a módulos como `expo-location`, `expo-camera` y las tareas de sincronización en segundo plano mediante `expo-task-manager` y `expo-background-task`.

La navegación está organizada en dos niveles:

- **Navegación de primer nivel (*tab bar*):** cinco pestañas inferiores (Inicio, Sesiones, Mapa, Sincronización, Perfil), visibles únicamente en las pantallas principales.
- **Navegación de pila (*stack*):** flujos de varios pasos como la creación de una nueva sesión o la captura de una observación, que se presentan sin el *tab bar*.

### 7.6.2. Gestión de estado global

El estado global de la aplicación se gestiona mediante el patrón **Context + useReducer** de React, sin dependencias externas de gestión de estado. El contexto `AppContext` expone el estado completo (sesiones activas, tipos de residuo, datos del usuario, estado de sincronización) y un *dispatcher* con acciones tipadas en TypeScript. Este patrón proporciona la predictibilidad de Redux con una superficie de código significativamente menor, adecuada para la escala del proyecto.

### 7.6.3. Base de datos local con SQLite

SQLite, integrado en Expo mediante `expo-sqlite`, actúa como base de datos local de la aplicación móvil. Se elige por ser una solución madura, ligera y suficientemente potente para almacenar el volumen de datos esperado durante una jornada de muestreo. Además, permite mantener control directo sobre el esquema local y sobre la cola de sincronización, reduciendo la complejidad respecto a incorporar una capa adicional de abstracción.

El esquema local replica el subconjunto del modelo de datos necesario para la operación en campo: `sampling_sessions`, `observations`, `litter_items` y `taxonomy_items`, junto con las tablas de apoyo de usuarios, grupos, campañas y catálogo (`environments` y `typologies`) que se descargan al iniciar sesión. Las operaciones pendientes de subida se registran en una tabla `outbox` (`sync_outbox`) que guarda, por cada cambio, el *endpoint* y el *payload* de la petición, el número de intentos, el último error y el estado de sincronización.

### 7.6.4. Flujo de captura de una observación

El flujo principal de la aplicación sigue esta secuencia:

1. El investigador inicia una nueva sesión de muestreo, especificando el nombre, el tipo de entorno y los parámetros del transecto.
2. Para cada ítem encontrado, abre la pantalla de nueva observación, selecciona el tipo de residuo a través del modal de taxonomía (organizado en cuadrícula visual por categorías e iconos), e introduce la cantidad, el tamaño, el color y comentarios.
3. Opcionalmente captura una foto geolocalizada del residuo.
4. Al finalizar la sesión de campo, revisa el resumen y puede iniciar la sincronización manualmente o dejar que se realice en segundo plano al recuperar cobertura de red.

El tamaño y el color se recogen en el formulario como ayuda a la caracterización del residuo, pero no se modelan como columnas propias. El ítem de residuo (`litter_items`) almacena la categoría, la cantidad y el material, y estas dos anotaciones se serializan dentro de su campo de notas, de donde la aplicación las vuelve a leer al editar la observación.

## 7.7. Sincronización offline

---

### 7.7.1. Patrón Outbox Queue

El mecanismo de sincronización se diseña siguiendo el patrón **Outbox Queue** (cola de salida), ampliamente utilizado en sistemas distribuidos con conectividad intermitente. El funcionamiento es el siguiente:

1. Cuando el usuario crea o modifica un registro (sesión u observación) en el dispositivo sin conexión, el registro se persiste en SQLite con estado **PENDING** y se inserta una entrada en la tabla **sync\_outbox** con el *payload* completo del cambio.
2. Cuando el dispositivo recupera conectividad, una tarea de sincronización en segundo plano o una acción manual del usuario lee la cola y realiza una petición **POST** `/api/sync/batch` con todos los registros pendientes.
3. El servidor procesa los registros en orden, los persiste en PostgreSQL y devuelve el estado canónico actualizado.
4. El cliente actualiza el estado de los registros procesados a **SYNCED** y descarga los cambios remotos realizados por otros usuarios mediante el *endpoint* **GET** `/api/sync/changes`, enviando la marca de tiempo de la última sincronización como parámetro de consulta.
5. Si un registro concreto falla, pasa al estado **FAILED** cuando los datos son inválidos o la subida no se completa, o a **CONFLICT** cuando el servidor detecta una versión más reciente. En ambos casos se notifica al usuario sin bloquear la sincronización del resto.

### 7.7.2. Estados de sincronización

Cada sesión y observación puede encontrarse en uno de cuatro estados, como muestra la tabla 7.3.

**Cuadro 7.3:** Estados de sincronización y su significado.

| Estado          | Significado                                                                                         |
|-----------------|-----------------------------------------------------------------------------------------------------|
| <b>PENDING</b>  | El registro existe solo en el dispositivo y está pendiente de subir al servidor.                    |
| <b>SYNCED</b>   | El servidor ha confirmado la recepción y persistencia del registro.                                 |
| <b>FAILED</b>   | La subida no se completó o el servidor rechazó el registro por datos inválidos. Requiere reintento. |
| <b>CONFLICT</b> | El servidor detectó una versión más reciente del registro. Requiere resolución.                     |

### 7.7.3. Resolución de conflictos

La estrategia de resolución de conflictos propuesta es **last-write-wins** con granularidad de registro: si dos usuarios modifican la misma sesión de forma independiente mientras están sin conexión, prevalece la versión con el **timestamp** más reciente. Esta estrategia es suficiente para el modelo de uso de OmniLitter, donde cada sesión de muestreo pertenece a un único observador responsable y los conflictos genuinos son infrecuentes.

#### 7.7.4. Garantía de identificadores únicos en entorno offline

Para que la estrategia Outbox Queue funcione sin depender de un identificador asignado por el servidor en el momento de la captura, los registros creados en modo offline usan **UUID v4 generados en el cliente**. Esto garantiza que las sesiones y observaciones creadas simultáneamente en distintos dispositivos no colisionarán al sincronizarse, independientemente del orden de llegada al servidor.

## 8. Desarrollo, implantación y validación

### 8.1. Organización del desarrollo

---

#### 8.1.1. Metodología de trabajo

#### 8.1.2. Gestión del proyecto en GitHub Projects

#### 8.1.3. Plan de pruebas

La validación del proyecto se organizó de forma incremental, siguiendo la misma división por sprints utilizada durante el desarrollo. El objetivo fue comprobar, en cada etapa, que las funcionalidades incorporadas no solo estaban disponibles desde la interfaz, sino que respetaban las reglas de negocio, los permisos, los contratos de datos y las restricciones definidas para el sistema.

La estrategia de pruebas combinó dos enfoques complementarios. Por un lado, se implementaron pruebas automatizadas sobre las partes más críticas del proyecto: esquemas compartidos, servicios de la API y determinadas utilidades del portal web. Por otro lado, los recorridos completos de usuario, especialmente aquellos relacionados con la aplicación móvil, el uso offline, la cámara, la ubicación y la sincronización, se validaron mediante pruebas manuales funcionales apoyadas en evidencias visuales.

Esta separación permite reflejar con precisión el alcance real de la validación. La memoria no atribuye al proyecto pruebas automatizadas que no existen, pero sí deja constancia de que las funcionalidades principales fueron comprobadas mediante el tipo de prueba más adecuado a su naturaleza.

**Cuadro 8.1:** Estrategia general de pruebas del proyecto

| Tipo de prueba                | Objetivo                                                                                    | Aplicación en OmniLitter                                                                                   |
|-------------------------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Pruebas de contra-to          | Verificar que los datos intercambiados entre clientes y API cumplen la estructura esperada. | Esquemas compartidos de autenticación, campañas, catálogo, sesiones, observaciones y fotografías.          |
| Pruebas unitarias de servicio | Comprobar reglas de negocio sin depender de la interfaz ni de una base de datos real.       | Servicios de usuarios, grupos, campañas, catálogo y observaciones del backend mediante mocks de Prisma.    |
| Pruebas automatizadas web     | Validar lógica auxiliar del portal web que no depende directamente de componentes visuales. | Cálculo de distancia de rutas y preferencias de exportación del dashboard.                                 |
| Pruebas manuales funcionales  | Comprobar recorridos completos desde la perspectiva del usuario final.                      | Portal web, aplicación móvil, captura de campo, uso offline, sincronización y evidencias visuales.         |
| Pruebas de regresión          | Comprobar que los cambios posteriores no rompen funcionalidades ya validadas.               | Reejecución de la suite automatizada y revisión manual de los flujos principales al cierre de cada sprint. |

La suite automatizada se ejecutó con *Vitest*. En el paquete compartido se utilizaron esquemas definidos con *Zod*, mientras que en el backend se validaron servicios mediante mocks de Prisma. Esta decisión permitió comprobar reglas de negocio relevantes sin depender de una base de datos PostgreSQL real ni de la ejecución completa de rutas HTTP.

**Cuadro 8.2:** Alcance real de las pruebas automatizadas

| Área   | Ficheros implementados            | Herramienta  | Cobertura principal                                                                               |
|--------|-----------------------------------|--------------|---------------------------------------------------------------------------------------------------|
| API    | Tests de <b>apps/api</b>          | Vitest       | Servicios de campañas, catálogo, grupos, usuarios y observaciones con mocks de Prisma.            |
| Web    | Tests de <b>apps/web</b>          | Vitest       | Preferencias de exportación del dashboard y cálculo de distancia de rutas.                        |
| Shared | Test de <b>packages/shared</b>    | Vitest + Zod | Esquemas compartidos de autenticación, campañas, catálogo, sesiones, observaciones y fotografías. |
| Mobile | Sin ficheros de test configurados | No aplica    | La aplicación móvil se validó mediante pruebas manuales funcionales y evidencias visuales.        |

Para facilitar la trazabilidad, las pruebas automatizadas se agruparon mediante códigos identificativos. Estos códigos se emplean en los apartados de cada sprint para relacionar las tareas desarrolladas con las comprobaciones realizadas.

**Cuadro 8.3:** Leyenda de códigos usados en las pruebas

| Código  | Significado                                                                                                                         |
|---------|-------------------------------------------------------------------------------------------------------------------------------------|
| SCH     | Pruebas de contrato sobre esquemas compartidos. Validan que los datos enviados por web, móvil o API cumplen la estructura esperada. |
| USR     | Pruebas del servicio de usuarios del backend.                                                                                       |
| GRP     | Pruebas del servicio de grupos y membresías del backend.                                                                            |
| CMP     | Pruebas del servicio de campañas del backend.                                                                                       |
| CAT     | Pruebas del catálogo científico usado por el sistema.                                                                               |
| OBS     | Pruebas del servicio de observaciones del backend.                                                                                  |
| WEB-SES | Pruebas de utilidades web relacionadas con sesiones y cálculo de distancias.                                                        |
| WEB-EXP | Pruebas de preferencias de exportación en el portal web.                                                                            |

La ejecución global de la suite automatizada finalizó correctamente. En total se ejecutaron 77 pruebas automatizadas distribuidas entre el paquete compartido, el portal web y la API.

**Cuadro 8.4:** Resultado global de la ejecución de pruebas automatizadas

| Paquete            | Resultado                            | Ficheros | Casos |
|--------------------|--------------------------------------|----------|-------|
| @omnilitter/shared | Correcto                             | 1        | 23    |
| @omnilitter/web    | Correcto                             | 2        | 15    |
| @omnilitter/api    | Correcto                             | 5        | 39    |
| @omnilitter/mobile | Sin tests automatizados configurados | 0        | 0     |
| Total ejecutado    | Correcto                             | 8        | 77    |

#### 8.1.4. Criterios de finalización de tareas

#### 8.1.5. Estructura de releases

## 8.2. Sprint 1: Base funcional del sistema

### 8.2.1. Objetivo del sprint

### 8.2.2. Parte del proyecto abordada

### 8.2.3. Tiempo invertido y reparto del trabajo

### 8.2.4. Implementación realizada

### 8.2.5. Pruebas realizadas

En el primer sprint se validó la base funcional del sistema. Las pruebas se centraron en autenticación, administración inicial de usuarios y grupos, campañas, catálogo, sesiones de muestreo y primeras observaciones. Esta fase era especialmente importante porque establecía los contratos y servicios sobre los que se apoyarían los sprints posteriores.

#### 8.2.5.1. Pruebas automatizadas implementadas

**Cuadro 8.5:** Pruebas automatizadas vinculadas al Sprint 1

| Tarea o funcionalidad                | Cobertura automatizada           | Comprobación principal                                                                                                                                            | Resultado |
|--------------------------------------|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Login, registro y cierre de sesión   | SCH-01, SCH-02                   | Validación del registro con contraseña robusta, confirmación coincidente y rechazo de contraseñas débiles o confirmaciones incorrectas.                           | Correcto  |
| Portal de administración de usuarios | USR-01, USR-03                   | Bloqueo de correos duplicados y creación de usuarios con o sin membresías iniciales.                                                                              | Correcto  |
| Gestión de grupos y membresías       | GRP-06, GRP-07                   | Restauración de grupos, permisos de administración y rechazo de acciones realizadas por usuarios sin autorización.                                                | Correcto  |
| Campañas y sesiones backend          | SCH-04 a SCH-13, CMP-01          | Estados válidos de campañas, referencias de catálogo y reglas de geometría para sesiones libres, por cuadrícula o por transecto.                                  | Correcto  |
| Catálogo científico                  | CAT-01 a CAT-08                  | Creación, actualización, detección de duplicados, borrado y restricciones por uso de entornos y tipologías.                                                       | Correcto  |
| API de observaciones y adjuntos      | SCH-14 a SCH-23, OBS-04 a OBS-06 | Observaciones con ítems obligatorios, coordenadas válidas, cantidades positivas, tipos de imagen soportados y control de acceso a observaciones propias o ajenas. | Correcto  |
| Formulario móvil de observación      | SCH-14 a SCH-16                  | Validación compartida de sesión, taxonomía, coordenadas, ítems y cantidades utilizada por el formulario móvil.                                                    | Correcto  |

Estas pruebas protegen reglas que no deben depender únicamente de la interfaz. Aunque los formularios ayuden a evitar datos incorrectos, los esquemas compartidos y los servicios de backend garantizan que la validación se mantiene ante datos reutilizados por distintos clientes o enviados directamente a la API.

#### **8.2.5.2. Pruebas manuales funcionales**

Las pruebas manuales de este sprint se reservan para los recorridos completos de usuario, como registro, inicio de sesión, administración básica, creación de campañas, creación de sesiones y registro inicial de observaciones desde la aplicación móvil. Su detalle se documenta mediante evidencias visuales en el apartado correspondiente de validación manual.

#### **8.2.6. Desviaciones e incidencias**

#### **8.2.7. Resultado del sprint**

## 8.3. Sprint 2: Captura de campo y análisis inicial

### 8.3.1. Objetivo del sprint

### 8.3.2. Parte del proyecto abordada

### 8.3.3. Tiempo invertido y reparto del trabajo

### 8.3.4. Implementación realizada

### 8.3.5. Pruebas realizadas

El segundo sprint reforzó la captura de datos de campo y el análisis inicial. La validación se orientó a comprobar la calidad del dato capturado, la incorporación de fotografías, el uso de ubicación GPS, el control de permisos sobre observaciones y las primeras funcionalidades de análisis del portal web.

#### 8.3.5.1. Pruebas automatizadas implementadas

**Cuadro 8.6:** Pruebas automatizadas vinculadas al Sprint 2

| Tarea o funcionalidad                   | Cobertura automatizada  | Comprobación principal                                                                                                    | Resultado |
|-----------------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------|-----------|
| GPS en sesión y observación             | WEB-SES-01 a WEB-SES-03 | Cálculo de distancia para rutas vacías, rutas con un único punto y suma de distancia Haversine entre puntos consecutivos. | Correcto  |
| Fotografías por observación y metadatos | SCH-17 a SCH-23         | Aceptación de formatos de imagen soportados y rechazo de tipos MIME no permitidos.                                        | Correcto  |
| Validación científica de observaciones  | SCH-14 a SCH-16         | Existencia de ítems, coordenadas válidas y cantidades positivas antes de aceptar una observación.                         | Correcto  |
| Permisos de observaciones               | OBS-01 a OBS-06         | Alcance de observaciones por usuario, visibilidad ampliada para gestores y bloqueo de observaciones ajenas sin permisos.  | Correcto  |
| Dashboard de métricas científicas       | WEB-EXP-01 a WEB-EXP-12 | Formatos soportados, persistencia de preferencias en <code>localStorage</code> y recuperación ante valores obsoletos.     | Correcto  |

La cobertura de este sprint resulta relevante para la calidad científica del sistema. Una observación no debe almacenarse sin ítems, con coordenadas imposibles, con cantidades no positivas o con ficheros de imagen no soportados. Además, las pruebas de permisos aseguran que cada usuario consulta únicamente la información que corresponde a su ámbito de autorización.

#### 8.3.5.2. Pruebas manuales funcionales

Las pruebas manuales de este sprint cubren los flujos que requieren interacción real con la aplicación móvil y revisión visual del resultado: permisos de ubicación, captura de

coordenadas, asociación de fotografías, validación de formularios, consulta por roles y visualización inicial de métricas en el portal web.

#### **8.3.6. Desviaciones e incidencias**

#### **8.3.7. Resultado del sprint**

## 8.4. Sprint 3: Sincronización, mapas y exportación científica

### 8.4.1. Objetivo del sprint

### 8.4.2. Parte del proyecto abordada

### 8.4.3. Tiempo invertido y reparto del trabajo

### 8.4.4. Implementación realizada

### 8.4.5. Pruebas realizadas

El tercer sprint estuvo orientado a sincronización, mapas y exportación científica. En esta fase fue necesario diferenciar con claridad entre las pruebas automatizadas existentes y la validación funcional de integración. La sincronización móvil-backend se comprobó mediante recorridos manuales completos, pero no se implementó una prueba automática de integración contra una base de datos real.

#### 8.4.5.1. Pruebas automatizadas implementadas

**Cuadro 8.7:** Pruebas automatizadas vinculadas al Sprint 3

| Tarea o funcionalidad                  | Cobertura automatizada            | Comprobación principal                                                                                                                   | Resultado |
|----------------------------------------|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Exportación CSV y soporte de rutas     | WEB-SES-01 a WEB-SES-03           | Cálculo de distancia de tracks como soporte para visualización, análisis de recorridos y exportación de información asociada a sesiones. | Correcto  |
| Preferencias de exportación científica | WEB-EXP-01 a WEB-EXP-12           | Formatos admitidos, persistencia de la preferencia de exportación y recuperación ante formatos no soportados.                            | Correcto  |
| Configuración y cambio de contraseña   | SCH-03, USR-04 a USR-06           | Rechazo de reutilización de contraseña, usuario inexistente, contraseña actual incorrecta y guardado seguro del nuevo hash.              | Correcto  |
| Sincronización móvil-backend           | Sin suite automatizada específica | La sincronización se validó mediante pruebas manuales de flujo completo. No se atribuye cobertura automatizada inexistente.              | Manual    |

La ausencia de una prueba automatizada específica para sincronización se considera una limitación de la validación automática. No obstante, se documenta de forma explícita para mantener la trazabilidad y evitar que la memoria presente como automatizada una comprobación que realmente fue funcional y manual.

#### 8.4.5.2. Pruebas manuales funcionales

Las pruebas manuales de este sprint son especialmente relevantes, ya que varias funcionalidades dependen de la interacción entre aplicación móvil, API y portal web. Incluyen tra-

bajo sin conexión, sincronización posterior, visualización en mapas, exportación científica y configuración de usuario.

#### **8.4.6. Desviaciones e incidencias**

#### **8.4.7. Resultado del sprint**

## 8.5. Sprint 4: Cierre funcional y refinamiento

### 8.5.1. Objetivo del sprint

### 8.5.2. Parte del proyecto abordada

### 8.5.3. Tiempo invertido y reparto del trabajo

### 8.5.4. Implementación realizada

### 8.5.5. Pruebas realizadas

El cuarto sprint correspondió al cierre funcional y al refinamiento del sistema. La validación tuvo un carácter principalmente regresivo: se reejecutaron las pruebas automatizadas existentes y se revisaron manualmente los flujos principales del portal web y de la aplicación móvil para comprobar que los cambios finales no introducían errores.

#### 8.5.5.1. Pruebas automatizadas implementadas

**Cuadro 8.8:** Pruebas automatizadas vinculadas al Sprint 4

| Tarea o funcionalidad                  | Cobertura automatizada                                                     | Comprobación principal                                                                                                                                        | Resultado |
|----------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Página de configuración web            | USR-02, USR-04 a USR-06, GRP-01 a GRP-15, CMP-02 a CMP-04, OBS-01 a OBS-06 | Permisos por rol, vista agregada de superadministrador, participación en campañas, archivado y restauración de grupos, membresías y alcance de observaciones. | Correcto  |
| Entorno en sesión y rangos científicos | CAT-01 a CAT-08, SCH-08 a SCH-13                                           | Validación de entornos, tipologías, referencias de catálogo y reglas compartidas que condicionan campañas y sesiones.                                         | Correcto  |
| Cierre funcional y refinamiento        | Suite completa                                                             | Reejecución de las pruebas de API, web y shared para comprobar que los ajustes finales no rompen funcionalidades previamente validadas.                       | Correcto  |
| Aplicación móvil                       | Sin suite automatizada específica                                          | Los ajustes visuales y funcionales de la aplicación móvil se validaron mediante recorridos manuales.                                                          | Manual    |

Esta validación permitió comprobar que las reglas principales del sistema seguían funcionando tras los cambios finales. Se mantiene como limitación que la aplicación móvil no dispone de una suite automatizada configurada.

#### 8.5.5.2. Pruebas manuales funcionales

Las pruebas manuales de este sprint se orientan a la revisión global del sistema: autenticación, configuración, campañas, sesiones, observaciones, catálogo, navegación del portal web y recorrido principal de la aplicación móvil.

### **8.5.6. Desviaciones e incidencias**

### **8.5.7. Resultado del sprint**

## 8.6. Resumen global del desarrollo

---

### 8.6.1. Funcionalidades implementadas

### 8.6.2. Evolución del sistema

### 8.6.3. Cobertura de requisitos

### 8.6.4. Matriz de trazabilidad prueba/requisito

### 8.6.5. Limitaciones finales

## 9. Despliegue y operación

### 9.1. Arquitectura de despliegue

---

### 9.2. Configuración de producción

---

### 9.3. Despliegue de API y base de datos

---

### 9.4. Despliegue del portal web

---

### 9.5. Build y distribución de la aplicación móvil

---

### 9.6. Limitaciones operativas

---

# 10. Conclusiones y trabajo futuro

10.1. Cumplimiento de objetivos

---

10.2. Limitaciones

---

10.3. Lecciones aprendidas

---

10.4. Líneas de trabajo futuro

---

# Bibliografía

- [1] Expo. Expo documentation. <https://docs.expo.dev>. Último acceso: mayo 2026.
- [2] L. Gallitelli, P. Girard, U. Andriolo, M. Liro, G. Suaria, et al. Monitoring macroplastics in aquatic and terrestrial ecosystems: Expert survey reveals visual and drone-based census as most effective techniques. *Science of The Total Environment*, 955:176528, 2024. doi:10.1016/j.scitotenv.2024.176528. Disponible en <https://www.sciencedirect.com/science/article/pii/S0048969724066841>. Último acceso: junio 2026.
- [3] Daniel Gonzalez, Georg Hanke, Gijsbert Tweehuysen, Bert Bellert, Marloes Holzhauer, Andreja Palatinus, Philipp Hohenblum, and Lex Oosterbaan. Riverine litter monitoring – options and recommendations. Technical Report EUR 28307 EN, Joint Research Centre, European Commission, 2016. doi:10.2788/461233. Disponible en <https://mcc.jrc.ec.europa.eu/documents/201703034325.pdf>. Último acceso: junio 2026.
- [4] Daniel Gonzalez-Fernandez and Georg Hanke. Toward a harmonized approach for monitoring of riverine floating macro litter inputs to the marine environment. *Frontiers in Marine Science*, 4, 2017. doi:10.3389/fmars.2017.00086. Disponible en <https://www.frontiersin.org/journals/marine-science/articles/10.3389/fmars.2017.00086/full>. Último acceso: junio 2026.
- [5] Camille Lacroix and Paul Vriend. Sharing best practices part 1: Monitoring techniques for macrolitter. OSPAR Riverine Litter Workshop, Temse, Belgium, 2025. Disponible en [https://www.ospar.org/site/assets/files/49581/annex\\_2\\_-\\_slides\\_on\\_monitoring\\_guidelines.pdf](https://www.ospar.org/site/assets/files/49581/annex_2_-_slides_on_monitoring_guidelines.pdf). Último acceso: junio 2026.
- [6] Meta Open Source. React documentation. <https://react.dev>. Último acceso: mayo 2026.
- [7] Meta Open Source. React native documentation. <https://reactnative.dev>. Último acceso: mayo 2026.
- [8] OpenJS Foundation. Express documentation. <https://expressjs.com>. Último acceso: mayo 2026.

- [9] Prisma Data, Inc. Prisma documentation. <https://www.prisma.io/docs>. Último acceso: mayo 2026.
- [10] SQLite Consortium. Sqlite documentation. <https://www.sqlite.org/docs.html>. Último acceso: mayo 2026.
- [11] The PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>. Último acceso: mayo 2026.
- [12] Vite. Vite documentation. <https://vite.dev>. Último acceso: mayo 2026.

# A. Declaración del uso de IA

En cumplimiento con las buenas prácticas académicas y de transparencia, se declara que para la elaboración de este trabajo se ha contado con el apoyo de herramientas de Inteligencia Artificial Generativa. El uso de dichas herramientas se ha limitado a tareas de apoyo, contraste y revisión, manteniendo los autores la responsabilidad final sobre las decisiones técnicas, el código entregado y la redacción definitiva de la memoria.

## A.1. Usos realizados

---

Las herramientas de IA se han utilizado principalmente en las siguientes tareas:

- **Organización inicial del trabajo.** Apoyo para estructurar la memoria, ordenar capítulos y decidir qué artefactos documentales eran necesarios.
- **Contraste de stack tecnológico.** Comparación de alternativas para el portal web, la aplicación móvil, el backend, la base de datos y el mecanismo de sincronización offline.
- **Prototipado de interfaces.** Ayuda en la generación y ajuste de mockups funcionales del portal web y de la aplicación móvil.
- **Diseño de arquitectura.** Apoyo en la elaboración de diagramas, separación de componentes, flujo de sincronización y modelo de datos.
- **Revisión de texto.** Reformulación y mejora de redacción de apartados de la memoria, siempre sobre contenido revisado por los autores.

El Cuadro A.1 resume las partes del trabajo donde se utilizó IA y con qué propósito.

**Cuadro A.1:** Detalle del uso de IA en el trabajo.

| Parte del trabajo     | Herramienta    | Propósito                                             |
|-----------------------|----------------|-------------------------------------------------------|
| Estructura de memoria | Claude / Codex | Proponer organización de capítulos y anexos.          |
| Stack tecnológico     | Claude / Codex | Comparar alternativas y razonar decisiones técnicas.  |
| Mockups funcionales   | Claude / Codex | Apoyar la generación y corrección de pantallas React. |
| Diseño del sistema    | Claude / Codex | Redactar borradores, diagramas y tablas técnicas.     |
| Revisión de redacción | Claude / Codex | Mejorar claridad, coherencia y estilo académico.      |

## A.2. Ejemplos de prompts utilizados

A continuación se recogen ejemplos representativos de prompts empleados durante el proyecto. No se incluyen todas las conversaciones, sino aquellos tipos de consulta que reflejan el uso realizado.

### Elección del stack

*Estamos diseñando una aplicación móvil de campo que debe funcionar sin conexión, capturar GPS y fotos, y sincronizar observaciones con un backend cuando recupere conectividad. Compara alternativas para la base de datos local y razona si tiene sentido usar SQLite directamente en este contexto.*

### Diseño de arquitectura

*Necesito plantear el capítulo de diseño de la solución para una plataforma formada por portal web, app móvil offline-first y backend REST. Propone una estructura para explicar arquitectura, componentes, base de datos y sincronización.*

### Revisión de requisitos

*Tenemos requisitos obtenidos en reuniones con el cliente y los hemos contrastado mediante mockups. Ayúdame a organizarlos en requisitos funcionales, no funcionales y de datos científicos, manteniendo trazabilidad con las reuniones.*

## Revisión de redacción

*He redactado un apartado de la memoria y quiero que suene más claro y técnicamente correcto. Reformula el texto manteniendo el contenido y las decisiones tomadas por el equipo.*

### A.3. Autoría y revisión crítica

---

Los autores declaran conocer y poder explicar el contenido incorporado al trabajo. Las respuestas obtenidas de las herramientas de IA se trataron como material de apoyo y fueron revisadas críticamente antes de su inclusión. La selección del dominio, la validación con el cliente, la toma de decisiones, la implementación final, la ejecución de pruebas y la entrega definitiva son responsabilidad de los autores.

## B. Requisitos detallados y validación con cliente

Este anexo recoge de forma más detallada la información utilizada para construir el registro de requisitos del proyecto y el proceso de validación seguido con el cliente. Su objetivo es dejar constancia del origen de las decisiones de diseño y de la evolución del alcance tras contrastar la propuesta inicial.

### B.1. Resumen de reuniones con el cliente

---

**Cuadro B.1:** Resumen de reuniones mantenidas con el cliente.

| Reunión                          | Objetivo                                                                                              | Resultado principal                                                                                                                                                              |
|----------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Toma inicial de requisitos       | Comprender el problema, el alcance esperado y las limitaciones de la herramienta existente.           | Se identificó la necesidad de una solución formada por aplicación móvil, portal web, funcionamiento offline, grupos de usuarios, mapas, dashboards y taxonomía de residuos.      |
| Revisión de requisitos y mockups | Contrastar los requisitos definidos con los prototipos interactivos del portal y la aplicación móvil. | Se validó que los mockups cubrían los procesos principales y se reforzó la prioridad de la captura offline, la separación por grupos y la trazabilidad de los datos científicos. |

### B.2. Requisitos obtenidos en la reunión inicial

---

Durante la primera reunión se extrajeron los siguientes requisitos de alto nivel:

1. Desarrollar una plataforma compuesta por aplicación móvil y portal web.
2. Permitir registrar residuos en distintos entornos naturales, no solo ríos y mares, sino también playas, bosques y otros terrenos.

3. Registrar información científica mínima de cada observación: tipo de residuo, cantidad, posición, fecha, hora, entorno y comentarios.
4. Permitir adjuntar fotografías a las observaciones.
5. Garantizar que la aplicación móvil pueda utilizarse sin conexión a internet.
6. Sincronizar automáticamente los datos cuando el dispositivo recupere conectividad.
7. Gestionar usuarios, grupos y permisos para mantener la privacidad de los datos de cada equipo.
8. Ofrecer en el portal web herramientas de consulta, mapas, dashboards y exportación de datos.
9. Mantener una taxonomía de residuos jerárquica, navegable y ampliable.
10. Preservar trazabilidad científica mediante versiones de taxonomía, metadatos de sesión y registros de auditoría.

### B.3. Validación de requisitos mediante mockups

---

La segunda reunión se centró en revisar los prototipos funcionales y comprobar que los flujos implementados representaban correctamente los requisitos ya recogidos. En lugar de generar un nuevo bloque de requisitos, esta revisión permitió confirmar la cobertura de los casos de uso principales:

- Inicio de sesión y diferenciación entre perfiles de administrador y colaborador.
- Selección de grupo activo y filtrado de la información asociada.
- Creación de sesiones de muestreo en la aplicación móvil.
- Registro de observaciones con taxonomía, cantidad, tamaño, color, comentarios, GPS y fotografía.
- Visualización de observaciones y sesiones desde el portal web.
- Consulta geográfica mediante mapas.
- Representación del estado de sincronización y de los datos pendientes.

Los ajustes visuales e internos realizados durante la construcción de los mockups se consideran decisiones de diseño del equipo y no requisitos adicionales solicitados por el cliente. Por ello no se documentan como cambios de alcance, sino como parte del proceso de refinamiento del prototipo.

## B.4. Requisitos funcionales detallados

---

El registro de requisitos funcionales queda agrupado en seis bloques:

- **Autenticación y sesión:** inicio de sesión, cierre de sesión y recuperación de contraseña.
- **Roles, grupos y permisos:** gestión de grupos, usuarios, membresías y selección de grupo activo.
- **Captura de campo:** sesiones de muestreo, observaciones, GPS, fotografías, cantidad, tamaño, color y comentarios.
- **Taxonomía:** navegación jerárquica, búsqueda y favoritos.
- **Sincronización:** funcionamiento offline, cola de sincronización, sincronización manual y automática, y estados por elemento.
- **Portal web:** listados, filtros, mapas, heatmaps, dashboards, fotografías, exportación, grupos, usuarios y configuración.

## B.5. Requisitos no funcionales detallados

---

Los requisitos no funcionales se agrupan en:

- **Seguridad:** comunicación cifrada, control de acceso por grupo y protección de adjuntos.
- **Fiabilidad:** sincronización reintentable y preservación de datos ante fallos o cierres inesperados.
- **Usabilidad:** interfaz adaptada a uso de campo en móvil y tablet.
- **Operación:** logs, copias de seguridad y matriz requisito-prueba.
- **Compatibilidad:** soporte para dispositivos móviles modernos, con especial atención a Android e iOS.

## B.6. Requisitos de datos científicos

---

Los requisitos de datos científicos se centran en garantizar que las observaciones registradas puedan analizarse posteriormente sin perder contexto:

- Taxonomía jerárquica con códigos estables.
- Metadatos de muestreo por sesión.
- Metadatos de foto cuando estén disponibles.

- Extensión versionada de la taxonomía.
- Validaciones de obligatoriedad, rango y consistencia.
- Exportaciones con diccionario de columnas y unidades.
- Versión de taxonomía y de aplicación asociada a sesiones/exportaciones.

## C. Manual de instalación y despliegue

## **D. Manual de usuario**

## **E. Pruebas completas**

## **F. API y diccionario de datos**

## G. Backlog completo